



DeSIST: Emergent security in IoT through Decentralized Strategic Interactions — A game-theoretic Zero Trust framework

Seyed Hossein Ahmadpanah^a, Meghdad Mirabi^b , Sanaz Sobhanloo^a,
Pania Afsharfarnia^a, Donya Fallah^a

^a Department of Computer Engineering, Technical and Vocational University (TVU), Tehran, Iran

^b Faculty of Computer Science, Technical University of Darmstadt, Darmstadt, Germany

ARTICLE INFO

Dataset link: <https://github.com/ahmadpanah/DeSIST>

Keywords:

Game theory
IoT security
Zero Trust Architecture
Decentralized systems

ABSTRACT

Numerous Internet of Things (IoT) devices are connecting our world, but they also introduce new security risks—particularly in large, power-constrained networks where traditional security techniques often fail. This paper proposes a new framework for IoT security, called DeSIST (Decentralized Strategic Interaction for Secure IoT), which is grounded in game theory and Zero Trust principles. Unlike approaches that rely on centralized watchdogs or explicit trust scores, DeSIST models interactions between IoT nodes as a sequence of strategic games. Treating nodes as rational agents, each maximizes its own expected utility when making decisions. Security emerges naturally because these games are designed with reward systems that encourage cooperation among trustworthy nodes while strategically isolating malicious or non-compliant actors. DeSIST employs a lightweight Local Information Assessor (LIA) to collect immediate, local, and contextually relevant information about ongoing or anticipated interactions, and a Strategic Decision Unit (SDU) to evaluate possible strategies and select the one that maximizes expected utility. Through theoretical analysis and extensive simulations, we show that DeSIST can decentralize and resource-efficiently uphold Zero Trust principles while significantly enhancing network resilience against common IoT attacks. Compared to existing approaches, the simulation results demonstrate notable improvements in both security and performance across different attack scenarios. DeSIST provides a promising path toward strong, incentive-driven, and emergent security in the evolving IoT landscape.

1. Introduction

By connecting an unprecedented number of devices, the Internet of Things (IoT) paradigm has sparked a revolutionary change in a variety of industries, from smart cities and industrial automation to healthcare and environmental monitoring [1–10]. However, the attack surface is inevitably increased by this widespread connectivity, leaving networks vulnerable to a wide range of security risks. Traditional security mechanisms, primarily designed for centralized and resource-rich environments, are largely insufficient in many IoT deployments due to their unique features, which include resource-constrained devices, physical accessibility, large-scale heterogeneous populations, and decentralized operational modes [11–15]. As a result, protecting these complex ecosystems from malevolent actors who could jeopardize user privacy, service availability, and data integrity continues to be a crucial and difficult task.

* Corresponding author.

E-mail addresses: Seyed.Hosseini.Ahmadpanah@gmail.com (S.H. Ahmadpanah), meghdad.mirabi@cs.tu-darmstadt.de (M. Mirabi), SanazSobhanloo@gmail.com (S. Sobhanloo), Paniaafsharfarnia@gmail.com (P. Afsharfarnia), donyafallah2004@gmail.com (D. Fallah).

<https://doi.org/10.1016/j.pmcj.2025.102124>

Received 11 June 2025; Received in revised form 10 October 2025; Accepted 22 October 2025

Available online 28 October 2025

1574-1192/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

The IoT presents serious challenges for current security paradigms. In distributed IoT topologies, perimeter-based defenses are essentially useless. Traditional Intrusion Detection and Prevention Systems (IDPS) frequently use signature databases that are ineffective against new attacks or impose computational and energy overheads that are inappropriate for devices with limited resources [16–18]. Although many advanced Trust Management Systems (TMS) have been proposed, using solutions ranging from fuzzy logic and blockchain to reputation scoring [19–23], they can introduce their own complications in terms of data aggregation, state maintenance, computational burden, and adaptability to dynamic threat landscapes. Furthermore, in scalable IoT systems, centralized enforcement points which are typical in some enterprise security models can turn into bottlenecks or single points of failure.

A strong foundation for contemporary security is provided by the Zero Trust Architecture (ZTA), which requires constant verification for each access request and makes no assumptions about implicit trust based on network location [24–27]. However, lightweight, flexible, and intrinsically decentralized mechanisms are required for its practical implementation in the IoT. A strong but little-studied basis for building such mechanisms is game theory, a mathematical framework for examining strategic interactions among rational decision-makers [28–32]. We postulate that carefully planned strategic interactions may result in robust security in the IoT, whereby individual nodes, acting in their own best interests, work together to increase the network's resilience and security. This is in contrast to approaches that mostly depend on direct enforcement actions, intricate trust calculations, or explicit detection.

We present the Decentralized Strategic Interaction for Secure IoT (DeSIST) framework to achieve this goal. By representing node-to-node interactions – such as packet forwarding, route selection in protocols like RPL [33], reporting anomalous behavior, or neighbor discovery – as a sequence of discrete strategic games, DeSIST rethinks IoT security. Each IoT node functions as a rational agent within this framework, choosing actions to optimize its expected utility, which can be characterized as balancing objectives such as risk mitigation, energy conservation, and successful data transmission. At the core of DeSIST lies the idea that, by carefully designing the payoff structures of these interaction games, it is possible to incentivize secure and cooperative behavior while naturally discouraging malicious or non-compliant actions through the strategic responses of other nodes. This effectively isolates harmful actors. The main contributions of this paper are as follows:

- (1) *A New Zero Trust Framework Based on Game Theory (DeSIST)*: To adhere to Zero Trust principles, we propose a comprehensive, decentralized framework in which security emerges from strategic interactions rather than relying solely on static trust mechanisms.
- (2) *Strategic Games for IoT Interactions*: We define specific game models with player and strategy characterizations, along with flexible payoff matrix design considerations, for key IoT functions such as packet forwarding, RPL parent selection, information reporting, and neighbor discovery.
- (3) *Lightweight Information Evaluation and Decision-Making Units*: We introduce a Strategic Decision Unit (SDU), which enables nodes to compute and select utility-maximizing actions based on predefined game structures, and Local Information Assessors (LIAs), which provide nodes with instantaneous, minimal-state contextual information.
- (4) *Strategic Isolation and Emergent Security Mechanism*: We demonstrate how DeSIST's game-theoretic foundations naturally encourage cooperative behavior while strategically isolating nodes that exhibit undesirable or malicious actions, thereby strengthening the network's overall resilience against prevalent IoT attack vectors.

The remainder of this paper is organized as follows: Section 2 reviews related work on various aspects of IoT security, establishing the context and motivation for DeSIST. Section 3 introduces the DeSIST framework, outlining its design goals, system architecture, and key components. Section 4 analyzes the emergence of DeSIST's security characteristics and evaluates its resilience against a range of attacks. Section 5 presents a game-theoretic analysis that provides a fundamental understanding of the strategic dynamics within DeSIST. Section 6 describes the experimental setup and reports the results of our simulations evaluating the performance of DeSIST. Finally, Section 7 concludes the paper and outlines directions for future research.

2. Background and related work

This section provides an overview of key topics relevant to IoT security, including common vulnerabilities, Zero Trust Architecture (ZTA), trust management, and game-theoretic security models. It then compares DeSIST with existing approaches and presents a gap analysis that positions the proposed framework within the broader research landscape.

2.1. Iot security vulnerabilities and attack vectors

The intrinsic characteristics of IoT environments – including limited CPU, memory, and energy resources; physical device exposure; reliance on wireless communication; heterogeneity; and large deployment scales – give rise to a unique range of vulnerabilities [34–38]. Numerous attacks targeting different layers of the IoT stack exploit these vulnerabilities. Routing attacks in Low-Power and Lossy Networks (LLNs) are particularly relevant to the interaction dynamics that DeSIST aims to model. The Routing Protocol for Low-Power and Lossy Networks (RPL), a standard for IPv6-based LLNs [33], is susceptible to various manipulation techniques.

A grayhole attack, for example, involves probabilistic or selective packet dropping. Malicious nodes may launch sinkhole or blackhole attacks by falsely advertising attractive routes – such as a very low rank – to lure traffic and subsequently drop or selectively forward packets [39]. To prevent the formation of Destination-Oriented Directed Acyclic Graphs (DODAGs), attackers

may also engage in rank attacks by manipulating their advertised RPL rank, isolating network segments or positioning themselves for eavesdropping [40]. Another disruptive tactic is the version number attack, which illegitimately increments the DODAG version number, triggering global re-convergence and resulting in significant control overhead and energy depletion [41,42].

Additionally, excessive transmission of RPL control messages – such as DIO, DAO, or DIS packets – can lead to control packet flooding, overwhelming legitimate nodes, depleting their resources, and destabilizing routing operations [43,44]. Beyond direct routing manipulations, nodes may also behave selfishly by deviating from protocol specifications to conserve their own resources. For instance, they may avoid forwarding packets or performing routing maintenance. While not overtly malicious, such behavior can be equally detrimental to network functionality [45,46].

Finally, the integrity and reliability of IoT applications can be compromised by False Data Injection (FDI) attacks, in which compromised nodes alter valid data or inject erroneous data during transmission [47,48]. DeSIST seeks to mitigate these behavioral threats by reshaping the strategic incentives for nodes, discouraging such actions through game-theoretic modeling.

2.2. Zero trust architectures (ZTA)

According to NIST [49], Zero Trust Architecture (ZTA) is a security paradigm that assumes no implicit trust is granted to assets or user accounts based solely on their network location or physical presence. ZTA enforces core principles such as micro-segmentation, least-privilege access, continuous verification of identity and device posture, comprehensive visibility, and dynamic policy enforcement. Every access request to enterprise resources must undergo rigorous, context-aware evaluation.

However, adapting ZTA to IoT environments presents distinct challenges due to factors such as device heterogeneity, limited computational and energy resources, and the large scale of deployments [50]. Traditional ZTA implementations in IoT often rely on centralized Policy Enforcement Points (PEPs), leveraging technologies like Network Function Virtualization (NFV) or Software-Defined Networking (SDN) to manage policies and control network flows [51,52]. While these approaches offer fine-grained control, they may also introduce single points of failure, increased latency, and scalability limitations.

Another common direction involves continuous monitoring and anomaly detection systems, which employ machine learning techniques to analyze device behavior and network traffic, identifying deviations from established norms [53,54]. Although effective in detecting threats, these mechanisms can impose substantial computational and communication overhead particularly burdensome for resource-constrained IoT devices.

DeSIST aligns with the ZTA principle of “*never trust, always verify*” by requiring nodes to strategically assess the utility of each interaction rather than relying on static trust assumptions. By embedding verification within the interaction dynamics themselves, DeSIST offers a decentralized and lightweight approach to implementing ZTA principles, making it more suitable for the unique constraints of IoT systems.

2.3. Trust management in IoT

Trust Management Systems (TMS) have been extensively studied as mechanisms to evaluate the dependability and integrity of nodes in decentralized networks. Among the most prominent approaches are reputation-based systems, such as the Beta-reputation model [55,56], which aggregate both first-hand observations and second-hand recommendations to compute a reputation score for each node. While these systems are adaptable and widely used, they incur overhead from maintaining reputation databases and are susceptible to manipulation tactics such as ballot stuffing and bad-mouthing, particularly when recommendations are not appropriately weighted [57].

Alternative trust modeling techniques include Dempster-Shafer Theory (DST) and Subjective Logic (SL), which explicitly represent belief, disbelief, and uncertainty [58]. These models are well-suited for environments where evidence is incomplete or ambiguous, but they often involve high computational complexity when combining multiple sources of information. Fuzzy logic-based trust systems offer flexibility by modeling imprecise trust notions through fuzzy sets and linguistic variables [59,60]; however, the definition of appropriate membership functions and inference rules typically requires significant domain expertise. Moreover, such systems can impose latency and scalability constraints on resource-constrained IoT devices.

In recent years, blockchain-based trust models have emerged, leveraging the immutability and transparency of distributed ledgers to record trust-related interactions and reputation scores [61–63]. These systems enhance auditability and reduce the risk of tampering, but they can also suffer from transaction costs and limited throughput.

Most conventional TMS aim to compute explicit trust scores or levels that guide decisions such as routing or access control. Despite their utility, these systems often require extensive data collection, processing, and storage. In contrast, DeSIST adopts a lightweight, implicit approach to trust. Rather than maintaining global or aggregated trust scores, DeSIST enables peers to continuously and locally reevaluate a node’s *trustworthiness* based on expected utility within specific game-theoretic interactions. This design reduces overhead and enhances adaptability in dynamic and resource-constrained IoT environments.

Table 1
Comparison of the DeSIST framework with related work.

Approach	Limitations	DeSIST improvements
<i>IoT security & Detection systems</i>		
Signature-based IDPS [16]	High computational overhead; ineffective against novel attacks	Game-theoretic interactions without signature reliance; utility-based decisions reduce overhead
Reputation-based TMS [19]	Vulnerable to ballot stuffing; requires global trust aggregation	Leverages LIA and decentralized game-theoretic decision-making
Blockchain-based TMS [61]	High latency and energy consumption; scalability concerns	Lightweight game framework with minimal state tracking using short-term memory
Fuzzy logic TMS [59]	Requires domain expertise; computationally intensive	Employs simplified heuristics based on probability estimation and lightweight rules
<i>Zero trust architecture</i>		
Centralized PEPs [51]	Single point of failure; increased latency in large-scale deployments	Fully decentralized architecture where each node makes independent decisions
Continuous monitoring [53]	High energy consumption and computational overhead	Event-driven feedback triggered only during interactions
Never trust principle [24]	Requires explicit verification mechanisms	Enables implicit verification through expected utility maximization
<i>Trust management systems</i>		
Dempster-Shafer theory [58]	Complex evidence fusion; high computational cost	Simplified estimation using local observations and short-term memory
Subjective logic [60]	Requires accurate belief propagation; scalability limitations	Updates local belief based on direct interactions (e.g., RSSI, ACK success)
<i>Game theory in security</i>		
Defender-Attacker games [68]	Assumes centralized game solvers; complex to implement	Decentralized approach with independent strategy computation using SDU
Bayesian games [70]	Computationally intensive; requires detailed probabilistic data	Heuristic LIA using minimal contextual data and low state overhead
Repeated games for routing [71]	Limited to single-layer routing and specific attacks	Multi-game framework (PFG, PSG, IRG, NDG) addressing diverse attack surfaces

2.4. Game theory in network security

Game theory provides a rigorous mathematical framework for analyzing strategic interactions involving cooperation and conflict among rational decision-makers [64,65]. Key concepts include players (decision-makers), strategies (available actions), payoffs (utilities resulting from strategy profiles), and solution concepts such as Nash Equilibrium [66], in which no player can unilaterally improve their payoff by deviating from their chosen strategy. Particularly relevant to security contexts are game-theoretic models such as signaling games, repeated games, and Bayesian games, which account for incomplete or asymmetric information.

This framework has been widely applied to address a range of network security challenges. For instance, defender-attacker models support optimal placement of intrusion detection systems and the strategic allocation of defensive resources [67–69]. Inspection games have been employed to simulate interactions between security inspectors and potential adversaries, capturing aspects of uncertainty and strategic deception [70].

Game-theoretic approaches have also guided the design of secure routing protocols, where nodes make path-selection decisions based on utility metrics such as reliability, latency, or energy efficiency—while anticipating adversarial behaviors [71]. Mechanisms that incentivize cooperative behavior in peer-to-peer and mobile ad hoc networks (MANETs) have been developed by structuring payoffs to reward cooperation and penalize selfish actions, such as refusing to forward packets or share resources [72].

In addition, specialized game-theoretic models have been proposed to mitigate specific security threats, including worm propagation, denial-of-service (DoS) attacks, and jamming [73,74]. These models enable the formulation of adaptive and robust security strategies that account for the strategic behavior of both legitimate and malicious nodes.

DeSIST builds upon these game-theoretic foundations by modeling interactions between IoT nodes as strategic, repeated engagements. Rather than relying on static trust scores or centralized enforcement, nodes dynamically assess the utility of each interaction based on outcomes observed over time. This allows the framework to adapt to evolving adversarial behaviors and environmental conditions while promoting cooperation and discouraging malicious conduct. By aligning with Zero Trust principles and maintaining low overhead, DeSIST demonstrates how game-theoretic mechanisms can foster emergent, decentralized security in dynamic IoT ecosystems.

2.5. Comparison with related work

Table 1 presents a comprehensive comparison between the DeSIST framework and existing approaches across four key IoT security domains.

Conventional IoT security solutions often exhibit inherent limitations that reduce their suitability for resource-constrained environments. For example, reputation-based trust management systems [19] are vulnerable to ballot stuffing attacks and require costly global trust aggregation. Signature-based intrusion detection and prevention systems (IDPS) [16] impose high computational overheads and lack effectiveness against zero-day or novel attacks. Similarly, blockchain-based approaches [61] suffer from substantial latency and energy consumption, which are particularly detrimental in IoT deployments.

Implementations of Zero Trust Architecture (ZTA) also face critical challenges. Centralized Policy Enforcement Points (PEPs) [51] introduce single points of failure and scalability bottlenecks, while continuous monitoring techniques [53] impose significant energy and computational overhead on constrained devices. In contrast, DeSIST overcomes these drawbacks by eliminating centralized components, reducing computational overhead through event-driven feedback, and enabling implicit verification via strategic decision-making.

Beyond Zero Trust mechanisms, another important research direction involves Trust Management Systems (TMS), which have been widely adopted to evaluate node reliability and foster cooperation in decentralized networks. However, conventional models – such as Dempster–Shafer Theory [58] and Subjective Logic [60] – require complex evidence fusion or accurate belief propagation, resulting in scalability and efficiency constraints. DeSIST, instead, establishes trust implicitly through repeated strategic interactions, allowing nodes to infer trustworthiness locally based on expected utility. This implicit, lightweight mechanism minimizes communication overhead and enhances adaptability in dynamic IoT environments.

Moreover, DeSIST's multi-game approach – encompassing Packet Forwarding Games (PFG), Parent Selection Games (PSG), Information Reporting Games (IRG), and Neighbor Discovery Game (NDG) – distinguishes it from other game-theoretic solutions [68, 70, 71], which often focus on single-layer interactions or rely on centralized game solvers. This holistic design enhances DeSIST's resilience against diverse attack vectors while addressing IoT resource limitations.

2.6. Gap analysis and DeSIST positioning

Despite the demonstrated utility of game theory in enhancing security, many existing works exhibit limitations that constrain their applicability as comprehensive IoT security frameworks. Commonly, game-theoretic models address isolated problems – such as routing or intrusion detection system (IDS) placement – rather than encompassing the diverse interaction types inherent to IoT environments. Furthermore, many approaches assume computational capacities beyond those available on resource-constrained IoT devices or rely on centralized entities for game parameter management and enforcement.

Much of the literature remains theoretical, with limited emphasis on lightweight, practical implementations suitable for deployment on IoT hardware. Approaches involving belief updates or Bayesian games often necessitate sophisticated probabilistic reasoning and extensive data collection, which are impractical for many IoT nodes.

DeSIST addresses these gaps by proposing a unified, lightweight framework that applies distinct game models to critical IoT interactions such as forwarding, routing, and reporting. Its design prioritizes decentralized decision-making, enabling each node to independently determine strategies based on minimal, local information. This facilitates emergent security, wherein globally optimal behavior arises from individual utility maximization rather than centralized enforcement or complex global state management.

A key enabler of this approach is Lightweight Information Assessment (LIA), which minimizes state retention at nodes, thereby improving feasibility in resource-limited settings. Through this approach, DeSIST harnesses the strategic power of game theory in a manner that is both practical for widespread IoT deployment and effective for enhancing security.

3. The DeSIST framework

This section presents the general philosophy underpinning the design of the DeSIST framework, outlines its core design goals, and describes its system architecture and key components.

3.1. Overall philosophy and design goals

Even IoT nodes with limited computational resources can be modeled as rational agents capable of making decisions to maximize their individual or collective utility. In DeSIST, security is treated as an *emergent property* – arising from the collective strategic behavior of these agents – rather than as a static state imposed by centralized authorities or derived from complex trust computations. Each node dynamically evaluates the risks and potential rewards of interacting with others before acting, thereby naturally adhering to the Zero Trust principle: “*never trust, always verify*”. This evaluation process serves as a game-theoretic verification of the anticipated outcomes of interactions.

The design of DeSIST is guided by four primary goals:

- **Decentralization:** By enabling nodes to make autonomous decisions, DeSIST eliminates the need for central controllers, thereby improving scalability and fault tolerance across the IoT network.
- **Lightweight Operation:** The framework is designed to operate within the resource constraints of IoT devices. It achieves this by leveraging real-time, context-aware information and selectively incorporating minimal, relevant historical data from previous interactions, rather than relying on extensive historical data aggregation.
- **Payoff Design Adaptability:** Security policies and preferences can be encoded into the system through flexible payoff structures. Even after the game-theoretic mechanisms are established, these payoff matrices can be adjusted to refine system behavior in response to evolving threats or policies.
- **Resilience:** By incentivizing cooperative behavior among legitimate nodes and strategically isolating malicious or non-cooperative actors, DeSIST enhances the robustness of the network against behavioral threats.

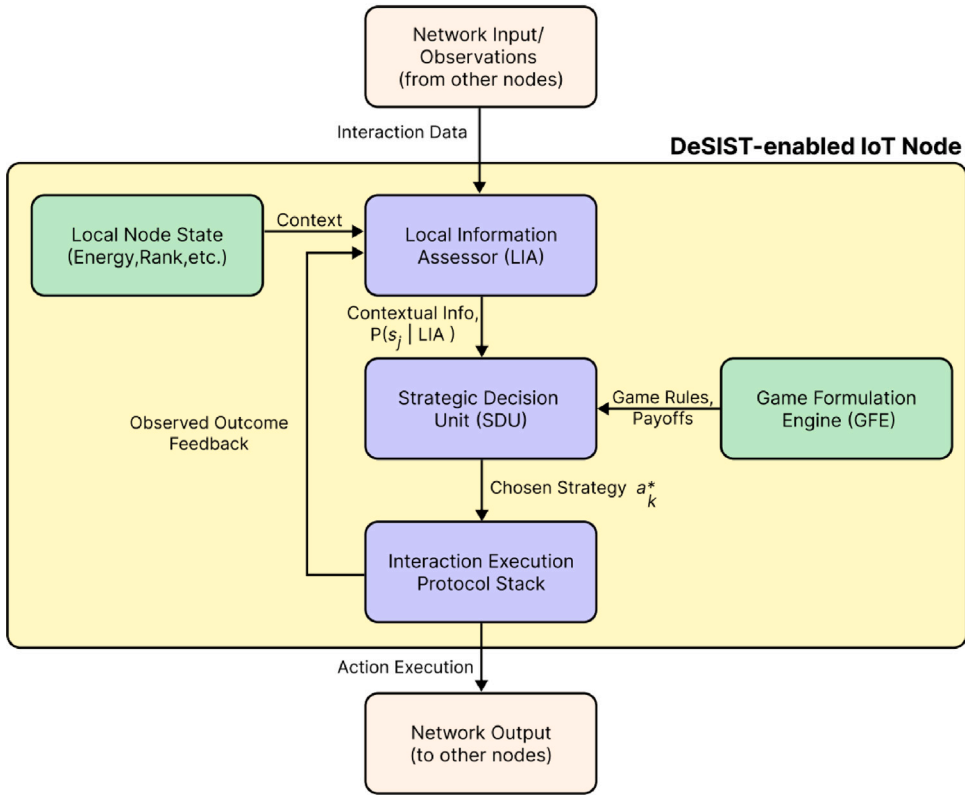


Fig. 1. System architecture of the DeSIST framework.

3.2. System architecture

Fig. 1 illustrates the system architecture of the DeSIST framework as implemented within each IoT node. The framework comprises several interrelated components:

- **Game Formulation Engine (GFE):** This component defines the structure of the interaction game and determines players, strategies, and associated payoff matrices for decision-making.
- **Local Information Assessor (LIA):** This component collects immediate, local, and contextually relevant information about ongoing or anticipated interactions, which serves as input for strategic analysis.
- **Strategic Decision Unit (SDU):** Acting as the node's decision-making core, this component processes input from the GFE and LIA to evaluate possible strategies and select the one that maximizes expected utility.
- **Interaction Execution Protocol Stack:** It executes the chosen strategy in a real-time interaction with another node and reports the observed outcome back to the LIA.

Following each interaction, the observed outcome is fed back into the LIA, allowing the system to continuously refine its future assessments and decisions through a feedback-driven learning loop.

Algorithm 1 illustrates the full decentralized interaction cycle among the components of the DeSIST framework. It captures the end-to-end process—from game formulation and local information assessment to strategy computation, interaction execution, and outcome-based learning.

3.3. Core components in the DeSIST framework

This section presents a detailed explanation of each core component of the DeSIST framework.

3.3.1. Game formulation engine (GFE) component

This component is responsible for establishing the *rules of engagement* for the various types of interactions that occur within an IoT network. It maintains a collection of predefined strategic games, specifying the players, their available strategies, and the payoff matrices that quantify the utility of all possible outcomes. These game definitions are either securely distributed during a trusted

Algorithm 1 Decentralized Strategic Interaction Process in DeSIST**Input:** Node i , Game Type $gameType$, Network State $networkState$, Historical Data $historicalData$ **Output:** Optimal Strategy a_i^* for Node i

```

1: function COMPUTEDESISTSTRATEGY( $i, gameType, networkState, historicalData$ )
   // Part 1: Game Formulation by GFE
2:    $GameStr \leftarrow \text{InitializeGame}(gameType, networkState, historicalData)$  ▷ Define the interaction scenario
   // Part 2: Local Information Assessment by LIA
3:   for each node  $j$  in  $GameStr.Players$  where  $j \neq i$  do
4:      $LIA\_context_j \leftarrow \text{GatherLocalInformation}(gameType, j, networkState)$ 
5:      $P_j \leftarrow \text{EstimateOpponentStrategyProbabilities}(gameType, LIA\_context_j, j)$ 
6:      $OpponentProbs[j] \leftarrow P_j$  ▷ Store probability estimates for opponent j
7:   end for
   // Part 3: Strategic Decision Making by SDU
8:    $a_i^* \leftarrow \text{CalculateOptimalStrategy}(i, GameStr, OpponentProbs)$  ▷ Select the optimal action
   // Part 4: Strategy Execution and Adaptive Feedback
9:    $Outcome \leftarrow \text{ExecuteStrategy}(a_i^*)$  ▷ Apply the chosen strategy and observe the result
10:   $\text{UpdateLocalKnowledge}(GameStr.Players, Outcome, gameType)$  ▷ Update knowledge base
11:  return  $a_i^*$ 
12: end function

```

network bootstrapping phase or pre-configured on the nodes. The logic for this component is encapsulated in the `InitializeGame` function of Algorithm 1.

The following are illustrative examples of strategic games formulated within the DeSIST framework to model key IoT interactions:

1. **Packet Forwarding Game (PFG):** The PFG models the fundamental process of data packet forwarding between nodes. A requesting node F_i (source or intermediate) interacts with a potential next-hop forwarder F_{i+1} . The strategies are:

$$S_{F_i} = \{SendPacketTo(F_{i+1}), HoldPacket\},$$

$$S_{F_{i+1}} = \{Cooperate(ForwardCorrectly), Defect(Drop/Modify/Delay)\}$$

The payoff function $U(F_i, action_{F_i}, action_{F_{i+1}})$ incentivizes cooperation: successful forwarding provides high utility to F_i and moderate rewards to F_{i+1} , while defection results in significant negative utility to F_i and limited selfish gain to F_{i+1} . Over time, repeated PFG interactions can promote *Tit-for-Tat* [75] strategies based on observed behavior.

2. **RPL Parent Selection Game (PSG):** The PSG models the interaction when a child node C selects a preferred parent from candidates $\{P_1, P_2, \dots, P_n\}$ based on DIO messages. The strategies are:

$$S_C = \{Select(P_k) \mid P_k \in \{P_1, \dots, P_n\}\},$$

$$S_{P_k} = \{HonestAdvertiser, DeceptiveAdvertiser\}$$

The payoff function $U(C, Select(P_k), action_{P_k})$ captures the utility from choosing a parent with reliable metrics (positive utility) or suffering from unreliable connections (negative utility). Node C uses observations from the LIA (e.g., metric consistency and stability) to infer parent reliability.

3. **Information Reporting Game (IRG):** The IRG captures the choice of an observing node O to report or ignore a potentially misbehaving node X , with R (e.g., gateway) as an indirect participant. The strategies are:

$$S_O = \{ReportMisbehavior(X), DoNotReport\},$$

$$S_X = \{ActuallyMisbehaving, BehavingNormally\}$$

The payoff function $U(O, action_O, action_X)$ reflects the trade-offs: reporting true misbehavior yields utility, false reports incur costs, and ignoring real misbehavior leads to degraded performance or penalties. The payoff function incentivizes accurate and honest anomaly reporting.

4. **Neighbor Discovery Game (NDG):** The NDG models the initial interaction between nodes A (advertising/responding) and B (discovering/initiating). The strategies are:

$$S_B = \{AttemptConnectionWith(A), Ignore(A)\},$$

$$S_A = \{AdvertiseHonestly, AdvertiseDeceptively\}$$

The payoff function $U(B, action_B, action_A)$ reflects the trade-offs: Connecting to a deceptive A results in negative utility (e.g., wasted resources), while connecting to an honest A offers positive outcomes. Although deception may yield temporary benefits, it leads to isolation. The LIA metrics such as probe responses and signal strength assist B in evaluating A 's behavior.

Algorithm 2 illustrates the GFE's operation, detailing the initialization process for different types of interaction games within the DeSIST framework. It receives the type of game, the current network state, and historical interaction data as inputs, and outputs a comprehensive game structure comprising the set of players, their possible strategies, and the calibrated payoff matrix.

Algorithm 2 GFE: Game Initialization Process

Input: Game Type *gameType*, Network State *networkState*, Historical Data *historicalData*
Output: Game Structure *GameStr* with players, strategies, and payoff matrix

```

1: function INITIALIZEGAME(gameType, networkState, historicalData)
2:   GameStr  $\leftarrow \emptyset$ 
3:   if gameType = PFG then ▷ Packet Forwarding Game
4:     GameStr.Players  $\leftarrow \{F_i, F_{i+1}\}$ 
5:     GameStr.Strategies[Fi]  $\leftarrow \{\text{SendPacketTo}(F_{i+1}), \text{HoldPacket}\}$ 
6:     GameStr.Strategies[Fi+1]  $\leftarrow \{\text{Cooperate}, \text{Defect}\}$ 
7:     GameStr.PayoffMatrix  $\leftarrow \text{CalibratePayoffs}(\text{PFG}, \text{networkState}, \text{historicalData})$ 
8:   else if gameType = PSG then ▷ Parent Selection Game
9:     GameStr.Players  $\leftarrow \{C\} \cup \{P_1, P_2, \dots, P_n\}$ 
10:    GameStr.Strategies[C]  $\leftarrow \{\text{Select}(P_k) \mid P_k \in \{P_1, \dots, P_n\}\}$ 
11:    for each parent Pk in  $\{P_1, \dots, P_n\}$  do
12:      GameStr.Strategies[Pk]  $\leftarrow \{\text{HonestAdvertiser}, \text{DeceptiveAdvertiser}\}$ 
13:    end for
14:    GameStr.PayoffMatrix  $\leftarrow \text{CalibratePayoffs}(\text{PSG}, \text{networkState}, \text{historicalData})$ 
15:   else if gameType = IRG then ▷ Information Reporting Game
16:     GameStr.Players  $\leftarrow \{O, X, R\}$ 
17:     GameStr.Strategies[O]  $\leftarrow \{\text{ReportMisbehavior}(X), \text{DoNotReport}\}$ 
18:     GameStr.Strategies[X]  $\leftarrow \{\text{ActuallyMisbehaving}, \text{BehavingNormally}\}$ 
19:     GameStr.Strategies[R]  $\leftarrow \{\text{TakeAction}, \text{IgnoreReport}\}$  ▷ Implicit strategy
20:     GameStr.PayoffMatrix  $\leftarrow \text{CalibratePayoffs}(\text{IRG}, \text{networkState}, \text{historicalData})$ 
21:   else if gameType = NDG then ▷ Neighbor Discovery Game
22:     GameStr.Players  $\leftarrow \{A, B\}$ 
23:     GameStr.Strategies[B]  $\leftarrow \{\text{AttemptConnectionWith}(A), \text{Ignore}(A)\}$ 
24:     GameStr.Strategies[A]  $\leftarrow \{\text{AdvertiseHonestly}, \text{AdvertiseDeceptively}\}$ 
25:     GameStr.PayoffMatrix  $\leftarrow \text{CalibratePayoffs}(\text{NDG}, \text{networkState}, \text{historicalData})$ 
26:   end if
27:   return GameStr
28: end function
29: function CALIBRATEPAYOFFS(gameType, networkState, historicalData)
30:   // Adjust Payoffs based on Current Network Conditions
31:   contextFactors  $\leftarrow \text{ExtractContextualFactors}(\text{networkState})$ 
32:   payoffMatrix  $\leftarrow \text{ApplyContextualAdjustments}(\text{contextFactors})$ 
33:   // Historical Performance-Based Calibration
34:   if historicalData contains interaction history for gameType then
35:     payoffMatrix  $\leftarrow \text{RefineWithHistory}(\text{payoffMatrix}, \text{historicalData})$ 
36:   end if
37:   return payoffMatrix
38: end function

```

Each game introduced in the DeSIST framework requires a carefully designed payoff matrix that reflects the goals, constraints, and contextual information relevant to its operation. However, statically assigning payoff values (e.g., rewards, penalties, or costs) is insufficient due to the dynamic and heterogeneous nature of IoT environments. Therefore, the payoff parameters in each game are dynamically adjusted based on the game context and the real-time state of the environment.

To support this dynamic adaptation, the DeSIST framework implements the *CalibratePayoffs* function within the GFE component. This function tailors the payoff parameters according to the game type and the contextual input received from the environment. As shown in Algorithm 2, the *CalibratePayoffs* function performs conditional tuning of payoff values for each supported game by incorporating historical interaction data and current network conditions. This process first extracts relevant contextual factors from the current network state and then applies rule-based adjustments to modify the base payoff entries accordingly. The adjusted payoff matrix therefore reflects the most recent operational conditions, ensuring that individual game outcomes contribute to maintaining global performance and stability. If historical interaction data for a given game type are available, the payoff matrix is further refined, as outlined in Algorithm 2. The resulting payoff configuration balances responsiveness with computational efficiency, providing DeSIST with a context-sensitive and economically viable mechanism for adaptive decision-making across distributed nodes.

Instead of using intricate optimization processes, the *CalibratePayoffs* function applies a succinct set of policy-driven rules to accomplish this calibration:

1. *Node-specific utility must be explicitly encoded:* Any node's behavioral disposition is mostly dictated by the weights assigned within its local payoff function, which is the triple $(w_{\text{succ}}, w_{\text{energy}}, w_{\text{risk}})$. The main levers that designers or network administrators can use to adjust a node's strategic posture without changing the underlying protocol logic are these scalar weights.

An energy-constrained sensor node that is powered by a limited-capacity battery serves as a tangible example. The node is set up with a significantly higher w_{energy} (i.e., $w_{\text{energy}} \gg w_{\text{succ}}, w_{\text{risk}}$). As a result, the node's utility calculus prioritizes energy conservation over cooperative forwarding right away. The node chooses shorter multi-hop routes or rejects specific forwarding requests when its residual energy drops below an established critical threshold because the expected utility of saving the remaining energy outweighs the penalty for temporary non-cooperation. This behavior is both locally and globally reasonable under the calibrated regime.

On the other hand, an inverse weight configuration is given to a medical-device gateway that is responsible for transmitting vital physiological data: w_{succ} is set to a very high value, while w_{energy} is purposefully suppressed. The likelihood of a successful packet delivery dominates the gateway's usefulness under this parametrization. In order to ensure timely and dependable data transfer despite its own resource limitations, it chooses the most dependable next-hop parents and is prepared to maintain higher transmission energy expenditure.

2. *Payoffs must evolve in response to network-wide context:* The payoff matrix's base entries, such as the reward for successful cooperation (R_s), the penalty for defection (P_L), the cost of transmission (C_T), and the reward for truthful misbehavior reporting ($R_{\text{TrueReport}}$), can be dynamically adjusted to reflect global network conditions, even though node-specific weights customize local behavior. Authenticated gateway broadcasts or locally deduced indicators cause these modifications, which guarantee that the strategic environment stays in line with operational demands in real time.

For example, the CalibratePayoffs function raises the defection penalty P_L and the truthful-reporting reward $R_{\text{TrueReport}}$ when it receives a gateway-issued alert indicating a notable increase in detected attacks. While the increased reward encourages watchful nodes to promptly report observed misbehavior, the increased penalty discourages logical adversaries from launching attacks. Without requiring extra computational or cryptographic overhead, the combined effect tightens the incentive structure and speeds up the isolation of malicious actors.

The function may dynamically increase the transmission cost C_T in a situation where there is severe network congestion. A higher C_T makes packet forwarding more expensive right away, which encourages nodes to choose less crowded paths or to reduce the frequency at which they transmit. This self-regulating mechanism maintains aggregate network stability and reduces congestion-induced performance degradation while remaining computationally light, making it appropriate for devices with limited resources.

The CalibratePayoffs function provides DeSIST with a responsive yet economical mechanism for strategic adaptation by methodically implementing these rule-based, policy-driven adjustments. By avoiding the computational and storage requirements of complex machine-learning models, the method ensures practical deployability in settings with limited processing cycles, memory, and energy.

The overall complexity of Algorithm 2 is determined by the processes involved in initializing players, their strategies, and the payoff matrix, including the dynamic calibration performed by the CalibratePayoffs function. For each individual game instance, the number of players and strategies is bounded and relatively small, so the initialization of players and strategy sets can be performed in constant time, $\mathcal{O}(1)$. Similarly, the CalibratePayoffs function, which extracts context factors, applies rule-based adjustments, and refines the payoff matrix using historical data, also operates in constant time per game instance, as these operations depend only on the fixed number of strategies and a small set of context metrics. Consequently, the total computational cost of initializing a single game remains $\mathcal{O}(1)$. When multiple games are instantiated across the network, the overall cost scales linearly with the number of game instances rather than with the total number of nodes, ensuring that both game initialization and payoff calibration remain lightweight and suitable for real-time adaptation in resource-constrained IoT environments.

3.3.2. Local information assessor (LIA) component

The primary sensory input used by an IoT node to make strategic decisions is provided by the Local Information Assessor (LIA) component. Its role is to obtain immediate, local, and contextually relevant information about ongoing or impending interactions without maintaining extensive historical records or performing complex aggregations. This focus on *minimal state* is critical for lightweight operation. The LIA's functionality is encapsulated in the GatherLocalInformation and EstimateOpponentStrategyProbabilities functions in Algorithm 1.

In its core design, the LIA distinguishes between benign network failures and malicious behavior. It directly observes interaction parameters, including protocol-level information such as the RPL rank (extracted from packet headers) and physical-layer metrics such as the Link Quality Indicator (LQI) and Received Signal Strength Indicator (RSSI). To complement these instantaneous observations, the LIA maintains a short-term memory that stores the most recent interactions with each neighbor, classifying a successful acknowledgment (ACK) as *Cooperate* and a failure as *Defect*.

The strength of the LIA lies in its contextual reasoning. For instance, the framework avoids *paranoia*: a packet lost over a link with low LQI or RSSI is not immediately interpreted as malicious behavior. Instead, the LIA may infer a benign failure caused by environmental factors such as interference, mobility, or energy-saving sleep schedules. Conversely, packet loss over a historically strong and stable link is interpreted as a stronger indication of defection. This contextual analysis prevents unjust penalties for well-behaved nodes operating under adverse network conditions. Additionally, the LIA integrates observations over a sliding window of interactions to smooth transient anomalies, reducing false positives due to isolated failures.

Algorithm 3 and Algorithm 4 together operationalize the LIA component's process for local information gathering and opponent behavior probability estimation.

Algorithm 3 describes the local information gathering process, which collects and maintains context-specific data for different interaction types within the DeSIST framework. For each interaction, the algorithm first gathers direct, game-specific observations for example: RSSI, LQI, and ACK status for the PFG; DIO consistency and rank fluctuations for the PSG; report accuracy and behavior patterns for the IRG; and signal strength for the NDG. It then updates the short-term memory by retaining only the latest five interactions, refreshes the node's operational state (energy level, RPL rank), and records authenticated broadcasts from neighbors.

Algorithm 3 LIA: Local Information Gathering

Input: Game Type $gameType$, Node j , Network State $networkState$
Output: LIA Context $LIA_context_j$ for Node j

```

1: function GATHERLOCALINFORMATION( $gameType, j, networkState$ )
   // Collect Game-Specific Observations
2:   if  $gameType == PFG$  then
3:      $LIA\_context_j.Observations[j].RSSI \leftarrow \text{GetRSSI}(networkState, j)$ 
4:      $LIA\_context_j.Observations[j].LQI \leftarrow \text{GetLQI}(networkState, j)$ 
5:      $LIA\_context_j.Observations[j].AckReceived \leftarrow \text{CheckAckStatus}(networkState, j)$ 
6:   else if  $gameType == PSG$  then
7:      $LIA\_context_j.Observations[j].DIOConsistency \leftarrow \text{CheckDIOConsistency}(networkState, j)$ 
8:      $LIA\_context_j.Observations[j].RankFluctuations \leftarrow \text{GetRankStability}(networkState, j)$ 
9:   else if  $gameType == IRG$  then
10:     $LIA\_context_j.Observations[j].ReportAccuracy \leftarrow \text{GetReportAccuracy}(networkState, j)$ 
11:     $LIA\_context_j.Observations[j].BehaviorPattern \leftarrow \text{AnalyzeBehaviorPattern}(networkState, j)$ 
12:   else if  $gameType == NDG$  then
13:     $LIA\_context_j.Observations[j].SignalStrength \leftarrow \text{GetSignalStrength}(networkState, j)$ 
14:   end if
   // Update Short-Term Memory and Node State
15:    $LIA\_context_j.ShortTermMemory[j] \leftarrow \text{GetRecentInteractions}(networkState, j, 5)$ 
16:    $LIA\_context_j.NodeState.energyLevel \leftarrow \text{GetEnergyLevel}(networkState, j)$ 
17:    $LIA\_context_j.NodeState.rank \leftarrow \text{GetRPLRank}(networkState, j)$ 
18:    $LIA\_context_j.AuthBroadcasts \leftarrow \text{GetValidatedAlerts}(networkState, j)$ 
19:   return  $LIA\_context_j$ 
20: end function

```

Most operations including observation collection, memory updates, and state refresh execute in constant time $\mathcal{O}(1)$. The only step that might involve iteration is processing incoming authenticated broadcasts; however, since the number of broadcasts per cycle is bounded by system design, this step is also effectively $\mathcal{O}(1)$. By combining immediate observations with limited historical context, the LIA delivers accurate, resource-efficient information to support real-time decision-making.

Algorithm 4 synthesizes the contextual information gathered by the LIA to estimate a probability distribution over all possible strategies for Node j in the given game type. The estimation process applies domain-specific, heuristic-based adjustments that integrate cross-layer observations, short-term interaction history, and authenticated external alerts, thereby avoiding oversimplified interpretations of failures and enabling context-aware decision-making under uncertainty.

Algorithm 4 LIA: Opponent Strategy Probability Estimation

Input: Game Type $gameType$, LIA Context $LIA_context_j$, Node j
Output: Strategy Probability Distribution P_j for Node j over All Possible Strategies for the Game Type $gameType$

```

1: function ESTIMATEOPPONENTSTRATEGYPROBABILITIES( $gameType, LIA\_context_j, j$ )
   // Initialize Strategy Probabilities based on Game Type
2:   if  $gameType == PFG$  then
3:      $P_j[Cooperate] \leftarrow 0.5$ 
4:      $P_j[Defect] \leftarrow 0.5$ 
5:   else if  $gameType == PSG$  then
6:      $P_j[HonestAdvertiser] \leftarrow 0.5$ 
7:      $P_j[DeceptiveAdvertiser] \leftarrow 0.5$ 
8:   else if  $gameType == IRG$  then
9:      $P_j[ActuallyMisbehaving] \leftarrow 0.3$ 
10:     $P_j[BehaveNormally] \leftarrow 0.7$ 
11:   else if  $gameType == NDG$  then
12:      $P_j[AdvertiseHonestly] \leftarrow 0.6$ 
13:      $P_j[AdvertiseDeceptively] \leftarrow 0.4$ 
14:   end if
   // Adjust Strategy Probabilities based on Observations
15:   if  $j \in LIA\_context_j.Observations$  then
16:      $obs \leftarrow LIA\_context_j.Observations[j]$ 
17:      $P_j \leftarrow \text{AdjustByObservations}(P_j, obs, gameType)$ 
18:   end if
   // Adjust based on Interaction History
19:   if  $j \in LIA\_context_j.ShortTermMemory$  then
20:      $history \leftarrow LIA\_context_j.ShortTermMemory[j]$ 
21:      $negativeRatio \leftarrow \text{CalculateNegativeRatio}(history)$ 
22:      $P_j \leftarrow \text{AdjustByHistory}(P_j, negativeRatio, gameType)$ 
23:   end if
   // Adjust based on Authenticated Alerts
24:   for each  $alert \in LIA\_context_j.AuthBroadcasts$  do
25:     if  $alert.targetNode == j$  then
26:        $P_j \leftarrow \text{AdjustByAlert}(P_j, gameType)$ 
27:     end if
28:   end for
29:    $P_j \leftarrow \text{NormalizeProbabilities}(P_j)$ 
30:   return  $P_j$ 
31: end function

```

In line with DeSIST's lightweight design philosophy, the algorithm employs constant-time $\mathcal{O}(1)$ heuristic rules instead of computationally expensive statistical or machine learning models. This ensures minimal processing overhead and supports real-time execution in resource-constrained environments. For example, an RSSI drop below a predefined threshold increases the probability of a defection strategy; inconsistencies in DIO messages raise the likelihood of deceptive advertising; repeated negative outcomes in short-term memory elevate suspicion of persistent misbehavior; and authenticated alerts from the network act as strong external indicators. Because the number of observations, history entries, and authenticated broadcasts per neighbor is bounded, all adjustment steps – including normalization of the fixed-length probability vector – execute in constant time, ensuring the system remains responsive and adaptive in dynamic and uncertain network conditions.

3.3.3. Strategic decision unit (SDU) component

The SDU is the decision-making core of DeSIST, responsible for identifying the strategy that maximizes the expected utility for each game instance. This process is implemented in the `CalculateOptimalStrategy` function of Algorithm 1.

Each node i operates according to a payoff function U_i , which quantifies its preferences over possible outcomes. This function encapsulates the node's role, objectives, and constraints. For instance, a *standard cooperative* node may use a payoff function of the form:

$$U_i = w_{succ} \cdot SuccessProb + w_{energy} \cdot EnergySaved - w_{risk} \cdot RiskExposure + w_{coop} \cdot FutureCoopBonus,$$

where *SuccessProb* reflects the likelihood of successful task completion (e.g., packet delivery), *EnergySaved* promotes energy efficiency, *RiskExposure* captures the potential for adverse outcomes, and *FutureCoopBonus* (a small value) rewards sustained cooperation. These preferences are governed by the weights w , which are typically pre-configured and represent the relative importance of each factor.

For example, an *energy-constrained* node may assign a significantly higher value to w_{energy} , leading it to favor strategies that conserve energy, even at the expense of slightly reduced task success. Conversely, a *critical-data* node would prioritize w_{succ} to maximize successful transmission. To anticipate attacker strategies, a defender may model the attacker's payoff function as one that favors disruption or information theft. Overall, the payoff function and its weights form the basis for rational strategy selection in diverse scenarios.

When selecting a strategy in a game defined by the GFE, the SDU performs the following steps:

1. Based on the game structure provided by the GFE and the estimated probabilities $P(s_j \mid LIA_context)$ of each opponent j 's strategy s_j as calculated by the LIA, the SDU computes the expected utility of each of its possible actions $a_k \in A_i$:

$$E[U_i(a_k)] = \sum_{\text{all possible opponent strategy profiles } s_{-i}} \left(\prod_{j \neq i} P(s_j \mid LIA_context) \right) \cdot U_i(a_k, s_{-i}), \quad (1)$$

where s_{-i} denotes the joint strategy profile of all other players. For two-player games, this simplifies to $E[U_i(a_k)] = \sum_{s_j \in S_j} P(s_j \mid LIA_context) \cdot U_i(a_k, s_j)$.

2. The SDU then selects the strategy a_k^* that maximizes the expected utility:

$$a_k^* = \arg \max_{a_k \in A_i} E[U_i(a_k)]. \quad (2)$$

Algorithm 5 formalizes the strategy selection process executed by the SDU. Instead of relying on advanced optimization solvers, the algorithm uses an utility calculation loop tailored for real-time use on constrained devices.

The SDU computes expected utilities by iterating over all possible opponent strategy combinations. Assuming independent opponents, the joint probability of each profile is the product of individual probabilities, reducing computational effort without significantly affecting accuracy.

The algorithm contains three nested operations: (i) iteration over the node's own strategies $|A_i|$, (ii) iteration over all joint opponent strategy profiles $|s_{-i}|$, and (iii) multiplication of probabilities across $n - 1$ opponents in each profile. Therefore, the worst-case time complexity is $\mathcal{O}(|A_i| \cdot (n - 1) \cdot \prod_{j \neq i} |S_j|)$, where n is the number of opponents.

For two-player games such as PFG, IRG, and NDG, $|A_i|$ and $|S_j|$ are small (typically two or three), so computation is effectively $\mathcal{O}(1)$, justifying the lightweight claim.

For the PSG, the child node must evaluate n candidate parents, each with two possible strategies (*HonestAdvertiser* or *DeceptiveAdvertiser*). Substituting $|A_i| = n$ and $|S_j| = 2$, the worst-case complexity becomes $\mathcal{O}(n \cdot (n - 1) \cdot 2^{n-1}) = \mathcal{O}(n^2 \cdot 2^n)$, and both computation time and memory (joint payoff matrix) scale exponentially. Hence, PSG is the only scenario in DeSIST where the SDU is not strictly $\mathcal{O}(1)$, and caution is advised for large numbers of parent candidates.

In general, the SDU remains lightweight and effectively constant-time for typical two-player games, while PSG represents a special case with exponential complexity.

Algorithm 5 SDU: Optimal Strategy Calculation**Input:** Node i , Game Structure $GameStr$, Opponent Probabilities $OpponentProbs$ **Output:** Optimal Action a_i^* for current node i

```

1: function CALCULATEOPTIMALSTRATEGY( $i$ ,  $GameStr$ ,  $OpponentProbs$ )
2:    $maxUtility \leftarrow -\infty$ 
3:    $a_i^* \leftarrow null$ 
4:   // Evaluate Each Possible Action for Node  $i$ 
5:   for each action  $a_i \in GameStr.Strategies[i]$  do
6:      $expectedUtility \leftarrow 0$ 
7:     // Generate All Possible Opponent Strategy Combinations
8:      $OpponentProfiles \leftarrow GenerateOpponentProfiles(GameStr, i)$ 
9:     for each profile  $s_{-i} \in OpponentProfiles$  do
10:       $jointProb \leftarrow 1.0$ 
11:      for each opponent  $j$  with strategy  $s_j$  in profile  $s_{-i}$  do
12:        if  $j \in OpponentProbs$  AND  $s_j \in OpponentProbs[j]$  then
13:           $jointProb \leftarrow jointProb \times OpponentProbs[j][s_j]$ 
14:        else
15:           $jointProb \leftarrow jointProb \times 0.5$  ▷ Default uniform probability
16:        end if
17:      end for
18:      // Get Payoff from Payoff Matrix and Add to Expected Utility
19:       $payoff \leftarrow GameStr.PayoffMatrix[a_i, s_{-i}]$ 
20:       $expectedUtility \leftarrow expectedUtility + (jointProb \times payoff)$ 
21:    end for
22:    // Update Optimal Action if This Action is Better
23:    if  $expectedUtility > maxUtility$  then
24:       $maxUtility \leftarrow expectedUtility$ 
25:       $a_i^* \leftarrow a_i$ 
26:    end if
27:  end for
28:  return  $a_i^*$ 
29: end function

```

3.3.4. Interaction execution protocol stack

After the SDU makes a decision, the node uses its normal processing and communication capabilities to execute the selected strategy in real-time interactions with other nodes and reports the observed outcomes back to the LIA. This process is implemented in the `ExecuteStrategy` and `UpdateLocalKnowledge` functions of Algorithm 1.

For example, in the PFG, a node may decide whether or not to forward a packet received from a neighbor. In the PSG, a node selects a parent from among potential upstream nodes in a routing protocol like RPL. In the IRG, the node may either report or ignore suspicious behavior observed in the network. In the NDG, the node manages neighbor discovery processes by sending or responding to probes or announcements.

The subsequent observation of the outcome of an executed interaction is a key task of DeSIST. The results of these observations are fed back to the node's LIA to update its local context. For example, in a PFG, if node i sends a packet to node j , the observations could include the link-layer RSSI and LQI values as well as whether an acknowledgment was received. In a PSG, if node C selects node P_k as its parent, the observations include the consistency of P_k 's DIO messages and any rank fluctuations, reflecting the stability of subsequent data transmissions through P_k . In an IRG, if node O reports potential misconduct by node X , the observations could include report accuracy metrics and identified behavior patterns. Finally, in an NDG, observations may include the signal strength of neighbor probes or the presence and validity of expected neighbor announcements. All of these observations are collected by the LIA and stored in the node's short-term memory and state to support future decision-making.

The LIA's minimal state regarding the parties involved is updated based on observed outcomes. This feedback loop is crucial for enabling adaptive strategic responses without the need to maintain intricate trust histories, as it allows the node to refine its probability estimates $P(s_j \mid LIA_context)$ for upcoming interactions. Through these direct consequences encountered in game-theoretic interactions, the system *learns* about the behavior of its peers.

4. Emergent security and attack resilience in the DeSIST framework

This section presents an analysis of the emergent security characteristics of the DeSIST framework, highlighting how its decentralized, game-theoretic design fosters robust defense mechanisms. We examine how strategic interactions among rational IoT nodes naturally lead to secure behaviors and demonstrate the framework's resilience against a wide spectrum of network attacks through adaptive, utility-driven decision-making.

4.1. Emergent security through strategic interactions

Within the DeSIST framework, security is not imposed via predefined trust scores or rigid security policies. Rather, it emerges organically from the decentralized, strategic decisions of rational IoT nodes that seek to maximize their utility. Two fundamental mechanisms drive the formation of secure behaviors in DeSIST: (1) the strategic isolation of defecting nodes whether malicious or non-compliant and (2) the incentivization of cooperation among legitimate participants.

4.1.1. Strategic isolation of defectors

In the DeSIST framework, nodes that persistently act maliciously or uncooperatively i.e., *defectors* in the game-theoretic sense become progressively marginalized within the network. This isolation arises primarily through *utility-driven avoidance*. When a node j repeatedly defects, for instance by dropping packets in a PFG or advertising false ranks in a PSG, the expected utility for other nodes to interact with node j declines sharply. Guided by the SDU component, rational nodes choose interaction strategies that maximize expected utility in each game instance. For example, if node F_i observes via its LIA that $P_{F_{i+1}}$ is low for cooperation in a PFG, it infers that routing through node F_{i+1} – a node prone to dropping packets – is suboptimal, and instead selects alternative forwarders.

This mechanism requires no global blacklists, which are often costly to maintain and verify. Instead, isolation emerges organically from numerous localized, autonomous decisions by individual nodes to limit or terminate interactions with defectors. As a result, defecting nodes gradually lose access to cooperative pathways, effectively pricing themselves out of beneficial network participation. Even if such nodes temporarily cause disruption, their capacity to exploit the network such as exfiltrating data through cooperative nodes or accessing legitimate services is severely constrained.

4.1.2. Motivation for cooperation

Within the DeSIST framework, cooperative behavior often emerges as the strategically dominant or utility-maximizing choice for rational, benign nodes whose payoff functions prioritize achieving their primary operational objectives, particularly in the context of repeated interactions. Across different strategic games in DeSIST, cooperation yields mutual benefits for legitimate participants, while defection imposes significant utility costs that can jeopardize future interactions. This interplay between positive incentives and negative consequences encourages nodes to adopt behaviors that sustain their role in the network.

The PFG illustrates this dynamic clearly. In the PFG, both participating nodes benefit from cooperation: node F_i achieves successful packet transmission to the next hop, while node F_{i+1} gains a cooperation reward or maintains a favorable reputation that supports future interactions by correctly forwarding the packet. Conversely, if node F_{i+1} defects by failing to forward the packet, node F_i incurs a significant utility loss.

During repeated interactions, node F_i 's SDU continuously updates the estimate $P_{F_{i+1}}$ (the probability that F_{i+1} cooperates) based on recent observations provided by the LIA. Nodes that persistently defect experience a rapid decline in this cooperation probability. As a result, if the expected utility of routing through node F_{i+1} falls below a defined threshold, rational neighbors proactively select alternative forwarders or may withhold transmissions altogether. This feedback mechanism naturally incentivizes nodes like F_{i+1} to cooperate in order to remain viable forwarders.

While the PFG serves as a concrete example, similar reciprocity and incentive principles extend across other strategic games within DeSIST, such as the PSG, the IRG, and the NDG. For instance, in the PSG, a parent node P_k that consistently provides stable routing (i.e., honest advertising) tends to yield higher utility for its children C , resulting in repeated selection. In the IRG, truthful participation is rewarded with privileges like gateway access or enhanced network conditions for accurate anomaly reporting. In the NDG, honest advertisers are motivated to maintain truthful announcements to preserve their reputation and maximize successful neighbor discoveries. Conversely, deceptive advertising leads to penalties such as reduced trust and diminished connectivity. These mechanisms collectively reinforce cooperation and trustworthiness throughout the network.

4.2. Attack resilience in the DeSIST framework

This section explains how the DeSIST framework addresses a variety of common and critical attacks through its game-theoretic, decentralized approach.

4.2.1. Blackhole, grayhole, and selective forwarding attacks

In these attacks, a malicious node M disrupts network functionality by dropping packets either all packets (blackhole), some packets (grayhole), or selectively chosen packets (selective forwarding). To counteract such behavior within the PFG, DeSIST implements a straightforward *Defect* detection mechanism. When the sending node S detects adverse outcomes, such as missing acknowledgments (ACKs), its LIA logs this behavior. Subsequently, the SDU lowers the expected utility of interacting with node M . Following one or a few failed transmission attempts, node S proactively selects alternative forwarders with higher expected utilities. This localized and rapid response effectively isolates the malicious node M from its immediate neighbors, mitigating its impact on the network.

4.2.2. Selfish nodes with abnormal energy conservation behavior

In this attack, a selfish node E conserves its own energy by refusing to forward packets for other nodes. Within the PFG, DeSIST treats this behavior as equivalent to a blackhole attack. Packet drops attributed to node E are recorded as defections by its neighbors, prompting them to avoid interactions with node E . Importantly, this self-serving strategy ultimately diminishes E 's utility, as reciprocal non-cooperation reduces its connectivity and impairs the forwarding of its own data. Consequently, DeSIST incentivizes node E to weigh short-term energy savings against the long-term benefits of cooperative network participation.

4.2.3. RPL rank and version number attacks

In these attacks, an adversary A attempts to disrupt routing by advertising fraudulent ranks or flooding the network with excessive RPL version number updates. The PSG is specifically designed to mitigate such threats. When a child node C 's LIA detects anomalies in DIO messages such as implausibly low ranks or rapidly increasing version numbers the SDU assigns a high probability that node A is a *DeceptiveAdvertiser*. Consequently, the expected utility of selecting A as a parent is significantly lowered, prompting node C to choose more stable and trustworthy parents. This strategy effectively curtails the attacker's ability to attract or retain child nodes, thereby neutralizing the attack.

4.2.4. DIO/DAO/DIS control message flooding attacks

In flooding attacks, a node overwhelms the network by transmitting excessive control packets, thereby depleting the resources of neighboring nodes. Although such attacks do not represent strictly pairwise interactions and are inherently difficult to counter directly, DeSIST provides indirect resilience against them. Within DeSIST, the LIA component detects abnormal message frequencies, which act as negative signals influencing other game decisions, such as reducing the likelihood of cooperation in the PFG. Additionally, nodes can report flooding behaviors through the IRG, facilitating collective, localized rate-limiting or other mitigation strategies that may be enforced by gateways.

4.2.5. Sybil attacks

Sybil attacks arise when an adversary generates multiple forged identities to gain disproportionate influence within the network. This represents a known limitation of the DeSIST framework, which treats each identity as an independent player and lacks a comprehensive identity management mechanism due to its lightweight design. As a result, a Sybil attacker can circumvent negative reputations by continuously creating new identities.

Mitigating Sybil attacks necessitates complementary solutions beyond the core game-theoretic approach. A promising strategy involves *resource-based entry costs* [76], requiring new identities to prove commitment such as forwarding a minimum number of packets or completing computational challenges (proof-of-work) prior to full participation. Furthermore, incorporating lightweight hardware-based attestation [77] can increase the difficulty of forging identities. Although these enhancements introduce additional system complexity, they constitute crucial directions for future research aimed at strengthening DeSIST's resilience against Sybil attacks.

4.2.6. Collusion attacks

Collusion attacks involve multiple malicious nodes coordinating their strategies to undermine the network, for example by forming a blackhole tunnel to selectively drop or intercept packets.

Since direct interactions with individual colluding nodes do not yield sufficient information to detect coordinated behavior, the current game models within DeSIST – focused on individual, non-cooperative decisions – are inherently limited in addressing this complex threat.

To effectively counter collusion, future extensions should move beyond modeling individual strategic choices to incorporate *coalition formation games* [78–80] and *cooperative game theory* [81–83]. These approaches would enable nodes to reason about collective tactics, potentially through the exchange of signed and validated reports concerning suspicious behaviors among neighbors. Formalizing *coalition-proof equilibria* [84–86] within this context presents a complex yet essential challenge for improving resilience. Moreover, ensuring that such advanced models remain computationally feasible for resource-constrained IoT devices is another important consideration.

At present, DeSIST's *Nash Equilibrium* analysis is limited to individual node interactions. Extending this framework to n -player games with multiple coordinated attackers remains an open research challenge. Formalizing *coalition-proof equilibria* in such multi-attacker settings represents a promising direction for future work.

4.2.7. Handling irrationality and non-malicious failures

While the DeSIST framework assumes rational actors, real-world nodes may exhibit irrational behavior such as deliberately causing disruption or experience faults and bugs resulting in non-malicious failures.

Despite its foundation on rationality, DeSIST's reactive mechanisms offer resilience against such irrational or faulty behaviors. Nodes exhibiting disruptive actions (e.g., erratic packet dropping) are treated as persistent defectors by their neighbors. Consequently, rational nodes avoid interactions with these disruptive nodes to maximize their own utility. Thus, regardless of whether the cause is irrationality, software faults, or intentional malice, DeSIST isolates nodes based on their adverse impact on the network.

To prevent premature isolation of well-behaving nodes affected by transient issues, the LIA integrates link quality metrics. This capability enables the framework to distinguish genuine malicious behavior from failures due to temporary channel conditions, thereby balancing robustness with fairness.

Table 2
Payoff matrix for the Packet Forwarding Game (PFG).

F_i 's Strategy (Sender)	F_{i+1} 's strategy (Forwarder)	
	Cooperate	Defect
Send packet	$(R_S - C_T, R_F - C_F)$	$(P_L - C_T, S_E)$
Hold packet	$(0, 0)$	$(0, 0)$

4.3. Adherence to zero trust principles

The DeSIST framework inherently integrates core principles of Zero Trust Architecture (ZTA) to enhance both security and resilience. This is realized through continuous verification, dynamic privilege management, and adaptive risk assessment based on real-time behavioral insights.

- *Never Trust, Always Verify*: No node is trusted by default. Each interaction is preceded by a strategic assessment of its expected utility, effectively asking, *Is it in my best interest to engage with this node right now?* This ongoing verification prevents blind trust.
- *Least Privilege*: Privileges, such as packet forwarding, are not granted based on identity alone but are earned through demonstrated utility. Nodes that behave poorly experience implicit revocation of their interaction privileges by their peers.
- *Assume Breach*: The DeSIST framework continuously assumes that any node could defect in the next interaction. Past cooperation does not guarantee future trust, maintaining a cautious posture throughout.
- *Continuous Re-Evaluation*: The utility and trustworthiness of interactions are dynamically re-assessed before every game instance, using the most recent data gathered by the LIA.
- *Reconciling Short-Term Memory with ZTA*: Although the LIA employs short-term memory to inform decisions, this does not conflict with Zero Trust principles. Rather than granting static trust based on historical attributes, DeSIST uses past behaviors as dynamic inputs for real-time risk assessment. Defection history acts as a behavioral health signal, which the SDU incorporates into expected utility calculations before each interaction. This approach parallels sophisticated ZTA systems that consider device health or recent activity in access decisions. Thus, LIA's memory supports verification rather than replaces it.

In general, DeSIST fosters a strategic environment where cooperative and secure actions naturally maximize utility for rational nodes, instead of relying on explicit trust constructs. Nodes that deviate from beneficial behaviors are progressively disadvantaged, making this decentralized, incentive-driven interaction model fundamental to the framework's attack resilience and security.

5. Theoretical game analysis of strategic interactions

This section presents a theoretical game-theoretic analysis aimed at providing a fundamental understanding of the strategic dynamics within DeSIST.

5.1. Theoretical analysis of the packet forwarding game (PFG)

To gain insight into the strategic interactions in the DeSIST framework, we analyze a simplified version of the PFG, involving two nodes: a sender F_i and a potential forwarder F_{i+1} .

At each stage of this repeated game, node F_i chooses between sending a packet to F_{i+1} (*SendPacketTo*) or withholding it (*HoldPacket*, which may imply sending to another node or delaying transmission). Simultaneously, node F_{i+1} decides whether to *Cooperate* by correctly forwarding the packet or to *Defect* by dropping or altering it. Table 2 illustrates the simplified payoff matrix, which represents F_i 's payoffs in a single stage, with F_{i+1} 's payoffs typically symmetric or adjusted for energy costs and rewards:

The parameters are defined as follows: R_S is the reward to F_i for a successful packet hop (e.g., +5); C_T is the transmission cost for F_i (e.g., -1); R_F is the reward for forwarding (e.g., +1); C_F is the forwarding cost for F_{i+1} (e.g., -1); P_L denotes the penalty for F_i due to packet loss (e.g., -10); and S_E is the energy saved by F_{i+1} when defecting (e.g., +0.5). The *HoldPacket* strategy yields zero utility, reflecting either delay or alternative transmission with comparable expected utility. The discount factor $\delta \in [0, 1)$ represents the weight given to future payoffs, with rational players maximizing their cumulative discounted returns in an indefinitely repeated game.

In a one-shot game, if $S_E > R_F - C_F$, a selfish F_{i+1} will prefer to *Defect*, resulting in suboptimal outcomes. However, in repeated interactions, strategies like *Tit-for-Tat*, which initiate cooperation and then mimic the opponent's previous move, can sustain cooperation. For cooperation to be a stable *Nash Equilibrium* supported by such strategies, the discounted future payoff from cooperating must exceed the one-time gain from defecting, formalized as:

$$\frac{R_F - C_F}{1 - \delta} \geq S_E$$

This analysis conceptually explains how cooperation can emerge and persist among rational nodes when future rewards are sufficiently valued and cooperation payoffs are appropriately balanced. While a full evolutionary game dynamics treatment or formal convergence proofs are beyond the scope of this paper, this serves as a foundation for further theoretical exploration.

Table 3
Payoff matrix for the Parent Selection Game (PSG).

C's strategy (Child)	P_k 's strategy (Parent)	
	Honest advertiser	Deceptive advertiser
Select	$(R_{Stable} - C_{Select}, R_{Gain} - C_{Maintain})$	$(P_{Unstable} - C_{Select}, G_{Deceit})$
Reject	(0, 0)	(0, 0)

Table 4
Payoff matrix for the Information Reporting Game (IRG).

O's strategy (Observer)	X's strategy (Subject)	
	Actually misbehaving	Behaving normally
Report misbehavior	$R_{TrueReport} - C_{Report}$	$P_{FalseReport} - C_{Report}$
Do not report	P_{Missed}	0

5.2. Theoretical analysis of the parent selection game (PSG)

To gain insight into the strategic interactions in the PSG, we examine a simplified, recurring exchange between a potential parent candidate P_k and a child node C seeking a parent. This game-theoretic model illustrates the cooperative defense of rational nodes against routing manipulation attacks.

At each decision point, the child node C must decide whether to *Select* P_k as its parent or to *Reject* it in favor of other candidates. Simultaneously, the prospective parent P_k chooses whether to behave as a *Deceptive Advertiser* (falsifying metrics, such as advertising an artificially low rank to appear more attractive) or an *Honest Advertiser* (reporting true routing metrics). Table 3 shows the payoff matrix for a single-stage interaction:

Parameters capture the strategic trade-offs for the parent and child nodes. Although there is a small selection cost C_{Select} , the main incentive for C is the reward R_{Stable} obtained from connecting to a stable, dependable parent. Choosing a dishonest parent carries a large penalty, $P_{Unstable}$, representing the negative effects of routing instability and packet loss. From the parent's perspective, being selected as a preferred parent results in a reward R_{Gain} and strengthens its place in the DODAG. The energy and processing costs $C_{Maintain}$ associated with honestly fulfilling its function balance this. The immediate gain from successful deception, G_{Deceit} , such as drawing traffic for a blackhole attack, drives malicious behavior. The *Reject* strategy yields a baseline utility of zero, as the child simply considers another candidate.

In a one-shot game, a malicious parent P_k is motivated to lie if the immediate gain from deception exceeds the honest reward (i.e., if $G_{Deceit} > R_{Gain} - C_{Maintain}$). Without prior information, a child node C could be deceived.

However, the PSG is a recurring game. The child's LIA evaluates the parent's performance after selection (e.g., node stability, consistency of subsequent DIOs). Poor performance quickly exposes a deceptive parent, increasing the likelihood of being labeled a *Deceptive Advertiser*. A rational child C will reject P_k in subsequent rounds, effectively separating the attacker.

For honest advertising to be a sustainable *Nash Equilibrium*, the long-term discounted payoff from maintaining a reliable reputation must exceed the one-time gain from deception. For the parent node P_k , this condition can be formalized using the discount factor $\delta \in [0, 1)$:

$$\frac{R_{Gain} - C_{Maintain}}{1 - \delta} \geq G_{Deceit}$$

This inequality indicates that rational nodes converge to an honest advertising strategy if future interactions are sufficiently valued (high δ) and the payoff structure appropriately favors long-term stability (R_{Gain}) over short-term malicious gains (G_{Deceit}). This analysis highlights how DeSIST's game-theoretic approach naturally promotes a robust and self-healing routing topology without centralized control.

5.3. Theoretical analysis of the information reporting game (IRG)

The IRG models a node's decision-making process when it observes potentially harmful activity. This game facilitates a community-based defense mechanism, designed to penalize false accusations, discourage indifference, and reward truthful reporting. We analyze the tactical choice of an observer node O after detecting an irregularity from a subject node X .

This situation is modeled as a decision problem under uncertainty for the observer node O . Uncertain about the actual state of the subject node X , which may be either *ActuallyMisbehaving* or *BehavingNormally*, the node O must choose between *ReportMisbehavior* and *DoNotReport*. The utility for O , based on its action and the actual strategy of X , is depicted in Table 4.

Several metrics influence the observer node's utility. A substantial reward, $R_{TrueReport}$, is granted for a correct report (true positive) that contributes to network security. The reporting process itself incurs an energy and processing cost, C_{Report} . False reports (false positives) incur a severe penalty, $P_{FalseReport}$, reflecting wasted resources and potential network disruption. Similarly, failing to report actual misconduct (false negatives) carries a penalty, P_{Missed} , representing potential harm from an unchecked malicious node. Correctly identifying and not reporting a benign node is the baseline neutral state, assigned a utility of zero.

Table 5

Payoff matrix for the Neighbor Discovery Game (NDG).

B's strategy (Discoverer)	A's strategy (Advertiser)	
	Advertise honestly	Advertise deceptively
Attempt connection	$(R_{HonestConnect} - C_{Connect}, R_{NewNeighbor} - C_{Connect})$	$(P_{DeceptiveConnect} - C_{Connect}, G_{Lure})$
Ignore	$(0, 0)$	$(0, 0)$

The true state of X is not known with certainty. Instead, the LIA estimates the probability p that X is misbehaving, making $(1 - p)$ the probability of normal behavior. The SDU selects the action that maximizes expected utility for the observer node O as follows:

$$E[U(ReportMisbehavior)] = p \cdot (R_{TrueReport} - C_{Report}) + (1 - p) \cdot (P_{FalseReport} - C_{Report}),$$

$$E[U(DoNotReport)] = p \cdot P_{Missed} + (1 - p) \cdot 0 = p \cdot P_{Missed}$$

The observer node O reports misbehavior if and only if:

$$E[U(ReportMisbehavior)] > E[U(DoNotReport)]$$

The DeSIST framework can derive the minimum probability threshold, p^* , for which reporting is the optimal strategy. Let $P_{FP} = -P_{FalseReport}$ be the positive cost of a false positive, and $P_M = -P_{Missed}$ the cost of a missed detection. The condition to report becomes:

$$p > \frac{P_{FP} + C_{Report}}{R_{TrueReport} + P_{FP} + P_M} = p^*$$

This analysis demonstrates that a rational, incentive-compatible threshold governs reporting decisions. The LIA's ability to produce accurate probability estimates p , together with the careful calibration of payoff values, allows the system's sensitivity to be adjusted. Increasing P_{FP} , for example, reduces spurious alerts but may overlook some threats, illustrating how the IRG supports a transparent and adjustable distributed anomaly detection mechanism.

5.4. Theoretical analysis of the neighbor discovery game (NDG)

The NDG models the crucial initial exchange between nodes, which serves as the foundation for all subsequent collaboration. It addresses the challenge of building trust in the absence of prior interactions. This analysis examines the strategic decisions of an advertiser node A and a discoverer node B .

Node B must choose whether to *AttemptConnectionWith* node A or to *Ignore* its advertisements during the discovery stage. Simultaneously, node A decides whether to *AdvertiseDeceptively* (e.g., by misrepresenting its capabilities) or *AdvertiseHonestly* (by providing accurate capability information). Table 5 shows the payoff matrix for a single-stage interaction.

The reward $R_{HonestConnect}$ is granted to the discoverer node B when it successfully establishes a reliable connection with an honest peer. This benefit is offset by a symmetric cost, $C_{Connect}$, which both nodes incur to complete the connection and handshake procedures. Conversely, node B suffers a penalty, $P_{DeceptiveConnect}$, representing lost resources and potential security risks if it connects with a malicious or dishonest peer. Node A gains $R_{NewNeighbor}$ from B for extending its connectivity and acquiring a legitimate neighbor. The primary incentive for a malicious advertiser is the potential gain from luring a victim, denoted by G_{Lure} . Finally, the *Ignore* strategy yields a neutral outcome with zero utility, as both nodes conserve resources and pursue alternative opportunities.

In a one-shot game, if $G_{Lure} > R_{NewNeighbor} - C_{Connect}$, a malicious advertiser A has a strong incentive to deceive, which may allow attacks such as Sybil infiltration.

DeSIST mitigates this risk through repeated interactions and the broader network context. Even if node B lacks a direct history with node A , its LIA can generate an initial probability estimate of A 's honesty based on passive observations, such as signal consistency. Subsequent interactions in other games (e.g., PFG or PSG) produce highly negative utility if node B is deceived, exposing node A as untrustworthy to both B and nearby observers.

The long-term cost for node A is isolation, as legitimate nodes will reject future connection attempts. For honest advertisement to be the dominant long-term strategy, the discounted value of maintaining a network of trustworthy peers must exceed the one-time benefit of a successful deception. Using the discount factor $\delta \in [0, 1)$, this condition can be formalized as:

$$\frac{R_{NewNeighbor} - C_{Connect}}{1 - \delta} \geq G_{Lure}$$

If the total benefits of participating in a trusted network are sufficiently high, this inequality holds. By balancing the long-term advantages of consistent, honest participation against short-term, high-risk gains, the NDG encourages a reliable network topology to emerge naturally from the first interaction.

6. Performance evaluation

This section presents the experimental setup, followed by the results of our simulations evaluating the performance of the DeSIST framework.

6.1. Experimental setup

To evaluate the performance of the DeSIST framework, we developed a custom discrete-event simulation environment in Python 3.12. Python was chosen for its flexibility in modeling complex game-theoretic interactions, diverse node behaviors, and varied network conditions, as well as for its extensive data analysis ecosystem (e.g., NumPy,¹ Pandas²). Several libraries supported the simulation: SimPy [87] handled resource management, process interactions, and event scheduling; NetworkX³ enabled topology generation (random, grid, scale-free), path computation, and network visualization; and NumPy facilitated numerical computations, statistical analyses, and probability estimation for the LIA or decision-making in the SDU. The simulator also incorporated realistic wireless communication models (e.g., probabilistic reception based on distance and interference, simplified MAC contention), energy consumption models, and attacker behavior models. It supported detailed implementation of DeSIST components, including game formulation in the GFE, information gathering and probability estimation in the LIA, strategy selection and utility calculation in the SDU, and protocol-stack-level interaction execution. This design enabled rapid prototyping and broad scenario exploration. The complete implementation of the DeSIST framework is publicly available.⁴

The simulated network model included the following features. Node counts varied from 30 to 100 to assess different densities. Nodes were randomly placed in a two-dimensional area to form multi-hop topologies; grid topologies were also used for targeted analyses. Communication was based on IEEE 802.15.4 MAC/PHY standards, using the Unit Disk Graph model initially, and Log-Normal Shadowing for more realistic runs. For LLN scenarios, the Routing Protocol for Low-Power and Lossy Networks (RPL) was employed. The energy model, proposed by [88], accounted for realistic power consumption in transmission, reception, idle listening, and CPU processing.

Each simulated node implemented the core DeSIST components. The GFE was preconfigured with game definitions, players, strategies, and payoff matrices for the PFG, PSG, IRG, and NDG. The LIA monitored ACK success or failure, parsed packet headers (e.g., source/destination, RPL DIO fields), and maintained minimal state from the most recent neighbor interactions. The SDU applied the utility maximization function using configurable utility profiles.

Payoff values for each game were explicitly defined. In the PFG, for example, node F_i might have $R_S = 5$, $C_T = 1$, $P_L = 10$, while F_{i+1} (selfish) had $R_F = 1$, $C_F = 1$, $S_E = 0.5$. Sensitivity analyses varied these parameters. Example utility weights for a cooperative node were $w_{succ} = 0.6$, $w_{energy} = 0.2$, $w_{risk} = 0.2$.

To ensure statistical significance and enhance generalizability under the specified experimental settings, all reported values are averaged over multiple simulation runs with randomized network topologies and node placements.

6.2. Simulation results under adversarial scenarios

To evaluate the performance of the DeSIST framework under adversarial scenarios, three attacker archetypes were modeled. A blackhole attacker joined the RPL DODAG (potentially advertising an attractive route) but dropped all data packets. A selfish node aimed to conserve energy, defecting in the PFG if $S_E > R_F - C_F$. An RPL Rank/Version Number attacker illegally manipulated the DODAG version or advertised a false rank. The proportion of attackers varied between 10% and 30%.

We compared DeSIST against three baseline approaches. A Standard RPL Network (no security) measured attack impact in an unprotected environment. An Ideal Scenario (no attackers) established upper bounds on PDR and performance. Finally, a Representative Trust-Based System (Representative TBS) for RPL (following principles from [89]) was used to compare DeSIST's game-theoretic approach against explicit trust calculation.

Performance metrics in adversarial scenarios were grouped as follows. Security metrics included the Malicious Node Isolation Index (MNII), i.e., the average number of legitimate nodes that ceased routing through a known malicious node after a fixed period, and the Packet Delivery Ratio (PDR), i.e., the ratio of successfully delivered packets to those generated. Network performance metrics included Average End-to-End (E2E) Delay, Network Throughput, and Network Lifetime/Average Energy Consumption (defined as the average per-node energy per unit time, or the time until a set fraction of nodes exhausted their energy). Overhead metrics included DODAG stability, i.e., the average number of parent changes per node per hour, and control overhead, measured as the ratio of control packets to data packets.

We considered the performance of the DeSIST framework against baseline approaches under various attack scenarios: (1) resilience to blackhole attacks, (2) mitigation of selfish node behavior, (3) resilience to RPL Rank/Version Number attacks, and (4) network lifetime under mixed attack conditions.

6.2.1. Scenario 1: Resilience to blackhole attacks

In this scenario, 20% of network nodes were configured as blackhole attackers, dropping all received data packets after advertising attractive RPL routes. Performance was assessed using three metrics: PDR, Average E2E Delay, and MNII. MNII ranges from 0 (no isolation) to 1 (complete isolation of malicious nodes from legitimate traffic paths).

As shown in Table 6, even under a substantial blackhole attack (20% attackers), DeSIST sustains a high PDR of 90.3%, significantly outperforming the Representative TBS (75.4%) and Standard RPL (35.2%). This resilience stems from the intrinsic

¹ <https://numpy.org/>

² <https://pandas.pydata.org/>

³ <https://networkx.org/>

⁴ <https://github.com/ahmadpanah/DeSIST>

Table 6
Performance comparison under blackhole attack (20% Attackers).

Approach	PDR (%)	Avg. E2E delay (ms)	MNII
Standard RPL (No security)	35.2 $\pm$ 2.5	2510 $\pm$ 150	0.00
Ideal scenario (No attackers)	98.1 $\pm$ 0.5	355 $\pm$ 20	N/A
Representative TBS	75.4 $\pm$ 1.8	645 $\pm$ 40	0.72 $\pm$ 0.05
DeSIST framework	90.3 $\pm$ 1.2	495 $\pm$ 30	0.91 $\pm$ 0.03

Table 7
Performance comparison with selfish nodes (25% Selfish, 70% Drop rate).

Approach	PDR (%)	Throughput (kbps)	Avg. energy (mJ/pkt)
Standard RPL (No security)	55.8 $\pm$ 2.1	5.6 $\pm$ 0.3	1.83 $\pm$ 0.10
Ideal scenario (No attackers)	98.1 $\pm$ 0.5	10.1 $\pm$ 0.2	0.51 $\pm$ 0.03
Representative TBS	78.2 $\pm$ 1.5	7.6 $\pm$ 0.2	1.05 $\pm$ 0.07
DeSIST framework	92.5 $\pm$ 1.0	9.1 $\pm$ 0.1	0.68 $\pm$ 0.04

Table 8
Performance comparison under RPL rank/version number attack (15% attackers).

Approach	PDR (%)	DODAG stability (Changes/hr)	Control over-head (%)
Standard RPL (No security)	44.6 $\pm$ 3.0	15.3 $\pm$ 1.2	0.41 $\pm$ 0.03
Ideal scenario (No attackers)	98.1 $\pm$ 0.5	1.6 $\pm$ 0.2	0.10 $\pm$ 0.01
Representative TBS	70.5 $\pm$ 2.2	5.1 $\pm$ 0.5	0.26 $\pm$ 0.02
DeSIST framework	88.7 $\pm$ 1.6	3.2 $\pm$ 0.3	0.17 $\pm$ 0.01

dynamics of the PFG: nodes rapidly assign low expected utility to interactions with identified blackhole nodes due to observed packet losses, yielding negative payoffs in the PFG and prompting strategic avoidance. The resulting MNII of 0.91 confirms effective isolation of malicious nodes. Although the average E2E delay (495 ms) is higher than in the Ideal Scenario (355 ms), it remains well below that of unprotected RPL, reflecting reduced retransmissions and efficient route adjustments. In this attack context, DeSIST's agile, utility-driven route switching demonstrates strong responsiveness to malicious behavior and dynamic network conditions.

6.2.2. Scenario 2: Mitigation of selfish node behavior

This scenario evaluated DeSIST's ability to mitigate the impact of selfish node behavior. Here, 25% of nodes were configured as selfish, each dropping 70% of packets they were supposed to forward in order to conserve energy. Performance was measured using PDR, total network throughput, and average energy consumed per successfully delivered packet.

As shown in Table 7, DeSIST achieved a PDR of 92.5% and a throughput of 9.1 kbps, substantially outperforming Standard RPL and exceeding the Representative TBS in both metrics. This performance is driven by the PFG dynamics, which rapidly assign low utility to unsuccessful forwarding interactions, prompting nodes to avoid selfish forwarders and favor more reliable routes. Consequently, delivery rates improve, and energy wasted on uncooperative nodes is minimized. The average energy per delivered packet (0.68 mJ) is markedly lower than in Standard RPL and competitive with TBS. Furthermore, if selfish nodes' payoff functions reward successful outgoing transmissions, reciprocal avoidance by other nodes can further diminish selfish nodes' own ability to transmit, reinforcing cooperative behavior.

6.2.3. Scenario 3: Resilience to RPL rank/version number attacks

This scenario assessed DeSIST's resilience against RPL Rank/Version Number attacks. In this configuration, 15% of nodes were designated as attackers that continuously advertised artificially low ranks to disrupt DODAG formation and attract traffic illicitly. Performance was evaluated using three metrics: PDR, DODAG stability (average number of parent changes per node per hour), and control overhead (ratio of control packets to data packets).

As shown in Table 8, DeSIST exhibits strong resilience to RPL Rank/Version Number attacks, achieving a PDR of 88.7% and improving DODAG stability to 3.2 changes/hr. This improvement is driven by the PSG's strategic routing decisions, which – guided by the LIA's detection of rank anomalies and abnormal advertisement patterns – reduce the likelihood of selecting deceptive parents. Compared to Standard RPL and Representative TBS, DeSIST maintains stable routes while keeping control overhead low (0.17). This efficiency arises from its reliance on localized decision-making, avoiding the excessive overhead of global trust exchanges or the large-scale re-convergence often triggered by attackers in other approaches.

6.2.4. Scenario 4: Network lifetime under mixed attack conditions

This scenario evaluated the long-term energy efficiency of the network under mixed attack conditions. 10% of the network nodes were configured as selfish, while another 10% acted as blackhole attackers. The primary performance metrics were the sustained PDR and network lifetime, defined as the time until 50% of legitimate nodes exhausted their energy resources.

Table 9

Performance comparison under mixed attack conditions (10% Blackhole + 10% Selfish).

Approach	Network lifetime	PDR (%)
Standard RPL (No security)	82 ± 5	41.3 ± 2.8
Ideal scenario (No attackers)	205 ± 8	98.0 ± 0.6
Representative TBS	138 ± 7	71.7 ± 2.0
DeSIST framework	176 ± 6	89.1 ± 1.5

Table 10

Performance comparison under irrational and intermittent node failures (10% Irrational + 10% Intermittent nodes).

Approach	PDR (%)	Avg. E2E delay (ms)	MIR (↑ better)	FIR (↓ better)
Standard RPL (No security)	48.6 ± 3.1	2410 ± 160	0.00	0.00
Representative TBS	73.2 ± 2.4	710 ± 50	0.81 ± 0.05	0.18 ± 0.04
DeSIST framework	88.9 ± 1.7	520 ± 30	0.87 ± 0.04	0.07 ± 0.02

As shown in Table 9, DeSIST substantially extends network lifetime, achieving 176 h until 50% node depletion, while maintaining a high PDR of 89.1% under mixed attack conditions. This improvement is due to DeSIST's strategic isolation of selfish and blackhole nodes, which reduces energy wasted on unnecessary retransmissions and interactions with uncooperative or malicious nodes. Compared to Standard RPL and the Representative TBS, DeSIST achieves superior long-term energy preservation, even accounting for the baseline energy costs of LIA and SDU computations.

6.3. Simulation results on resilience to irrational and intermittently failing nodes

To validate DeSIST's ability to handle nodes exhibiting irrational or benign-but-faulty behaviors, we extended our simulation framework to include two additional operational models. The first model introduced irrational nodes, where a subset of the network was configured to randomly drop packets with a fixed probability of 50%, irrespective of any strategic payoff or utility calculation, thereby mimicking buggy or unpredictable device behavior. The second model simulated benign nodes with intermittent failures, where otherwise cooperative nodes experienced temporary link degradation due to factors such as signal fading or mobility. In these cases, packet losses were not intentional but were instead caused by transiently reduced RSSI/LQI values over random intervals of 10 to 20 s.

The experimental setup consisted of a 60-node random topology in which 10% of nodes were irrational and another 10% were benign nodes with intermittent failures. Performance was evaluated over 500-second simulations, with results averaged across 10 repetitions. We compared the DeSIST framework against the Standard RPL and the Representative TBS. Key performance metrics included the overall Packet Delivery Ratio (PDR), the Average End-to-End (E2E) Delay, the Malicious Isolation Rate (MIR)—representing the fraction of irrational or faulty nodes correctly isolated – and the False Isolation Rate (FIR) – denoting the fraction of benign nodes mistakenly isolated.

As shown in Table 10, the results demonstrate DeSIST's superior resilience and discernment in mixed fault conditions. In terms of threat neutralization, DeSIST correctly isolated approximately 87% of the irrational nodes, achieving an MIR comparable to the Representative TBS (81%) while showing greater robustness against unpredictable behaviors. The most significant improvement, however, lies in the mitigation of collateral isolation. DeSIST maintained a very low FIR of 7%, substantially outperforming the Representative TBS (18%), which highlights its ability to preserve benign nodes during transient link disruptions.

This performance gain stems from the LIA's contextual reasoning, which combines RSSI/LQI indicators with a sliding memory window to distinguish temporary, environment-induced failures from persistent malicious or irrational activity. As a result, DeSIST minimizes false isolation and sustains network connectivity. Consequently, the framework achieves a high PDR of 88.9%, compared to 73.2% for the Representative TBS and only 48.6% for the Standard RPL. This demonstrates that the low collateral isolation rate does not compromise network throughput. Furthermore, DeSIST attains a lower average E2E delay (520 ms) than the Representative TBS (710 ms), since fewer benign nodes are mistakenly excluded from active routing paths, ensuring efficient data forwarding under challenging conditions.

6.4. Simulation results on sybil attack resilience with resource-based entry cost

To address the known susceptibility of DeSIST to Sybil attacks, we implemented and evaluated a resource-based entry cost mechanism, transitioning the proposed mitigation from theoretical concept to empirical validation. Specifically, a *Proof-of-Cooperation* (PoC) entry cost was introduced into the simulation environment. Under this mechanism, a newly joining node enters a probationary state in which it can receive traffic and participate in route discovery, but cannot be selected as the primary parent for critical data. To transition to a verified state, the probationary node must successfully forward $N = 50$ packets on behalf of its neighbors, incurring a direct time and energy cost for establishing a functional identity.

Table 11
Performance comparison under a sybil-assisted blackhole attack (20 Sybil nodes).

Approach	PDR (%)	MTAR (%)	Sybil NVR (%)
Ideal scenario (No attackers)	98.2 $\pm$ 0.6	0.0	N/A
Standard DeSIST (no entry cost)	21.5 $\pm$ 3.1	78.3 $\pm$ 4.5	100.0 (instant)
DeSIST + PoC entry cost	89.8 $\pm$ 1.8	5.1 $\pm$ 1.2	0.0

Table 12
Computational and state overhead comparison.

Approach	Component	Computational ops (Count)	State size (Bytes/Neighbor)	Model size (Parameters/Rules)
DeSIST framework	LIA estimation	~ 15	~ 8	4–5 Rules
	SDU maximization (2-player game, e.g., PFG)	~ 20	0	4 payoff values
	Total DeSIST cycle (2-player game)	~ 35	~ 8	~ 10 Total
	SDU maximization (PSG for n candidates)	$\sim 20 \times n$	0	4 payoff values
	Total DeSIST cycle (PSG for n candidates)	$\sim 15 + (20 \times n)$	~ 8	~ 10 Total
Representative TBS	Beta reputation update	~ 60	16	1 formula
ML-based anomaly detection	MLP inference ($10 \times 5 \times 1$)	~ 1500	240	61 parameters

We tested this mechanism against a coordinated Sybil-assisted blackhole attack in a 100-node network, where an adversary attempted to introduce 20 Sybil identities. Performance was evaluated using three metrics: the Sybil Node Verification Rate (NVR), i.e., the percentage of malicious identities that satisfied the PoC requirement; the Packet Delivery Ratio (PDR), i.e., the ratio of successfully delivered packets to those generated; and the Malicious Traffic Attraction Rate (MTAR), i.e., the proportion of network traffic diverted to malicious nodes. To establish baselines, we compared Standard DeSIST (no entry cost) and DeSIST enhanced with PoC against an ideal scenario with no attackers.

The results in Table 11 demonstrate the effectiveness of the PoC entry cost in mitigating Sybil attacks. Without the entry cost, all 20 Sybil nodes are immediately recognized as valid participants, attract 78.3% of network traffic, and reduce the PDR to just 21.5%, underscoring the severity of this vulnerability. In contrast, when the PoC mechanism is enabled, Sybil nodes remain probationary and fail to meet the forwarding requirement, resulting in a 0.0% verification rate. Consequently, they attract only 5.1% of traffic (primarily initial discovery probes), and the network sustains a high PDR of 89.8%, close to the ideal case.

Although the PoC mechanism introduces a short initialization latency for new legitimate nodes, the security benefits are substantial. This experiment confirms that the resource-based entry cost provides a practical and highly effective enhancement to the DeSIST framework, converting a critical weakness into a quantifiable defense.

6.5. Simulation results on computational overhead

To quantitatively validate the claim that DeSIST is a lightweight framework, we estimated its computational and state overhead using a simulation-based analysis in our Python discrete-event environment. Two representative security mechanisms were used as baselines: a lightweight trust-based system (Representative TBS) using a Beta reputation model [89], and a more resource-intensive machine learning system (ML-based Anomaly Detection) employing a basic Multi-Layer Perceptron (MLP).

Three hardware-agnostic metrics were measured for a single decision-making instance of each approach: (1) the number of basic arithmetic operations, as a proxy for CPU execution time; (2) state size per neighbor, representing dynamic RAM usage; and (3) model size or rule complexity, as a proxy for Flash/ROM memory footprint.

Table 12 highlights that DeSIST imposes very low computational and memory overhead, supporting its suitability for resource-constrained IoT devices. A full decision cycle of DeSIST, using the 2-player game (PFG), requires approximately 35 basic arithmetic operations. This includes 15 operations for the LIA estimation, which performs simple heuristic comparisons of RSSI/LQI and recent interaction history, and 20 operations for SDU's simple utility maximization over the 2×2 payoff matrix of the PFG. In contrast, the Representative TBS requires roughly 60 operations per reputation update, and the ML-based anomaly detection approach requires over 1500 operations per inference, emphasizing the efficiency of DeSIST for high-frequency decisions.

For multi-candidate scenarios, the PSG (n -player game) is considered. Here, the computational cost of SDU's simple utility maximization scales linearly with the number of candidates, n . Each candidate adds approximately 20 operations for SDU, in addition to the 15 operations for LIA estimation. The total number of operations for a PSG instance is therefore approximately $15 + (20 \times n)$. This scaling remains practical for small to moderate candidate sets in typical IoT networks and allows DeSIST to handle multi-candidate decisions efficiently without introducing prohibitive computational complexity.

State size requirements are similarly modest. The LIA maintains only 8 bytes per neighbor to store recent interactions and link quality metrics, while SDU's simple utility maximization does not add any additional per-neighbor state. The total model size, including the heuristic rules of LIA and payoff values of SDU, is approximately 10 items, both in the PFG and PSG cases. By comparison, the Representative TBS stores 16 bytes per neighbor for reputation parameters, and the MLP-based approach incurs a substantial fixed memory cost of 240 bytes, even for a small network.

These results confirm that DeSIST achieves a favorable balance between security and efficiency. Its low computational, state, and model overheads make it highly suitable for deployment on constrained IoT devices, including scenarios with multiple candidates where PSG scaling is needed.

Table 13

LIA detection accuracy across different network conditions.

Network condition	TP	FP	TN	FN	Precision	Recall	F1-score
High signal quality (RSSI > -70 dBm)	892	23	1456	11	97.5%	98.8%	98.1%
Medium signal quality (-80 < RSSI < -70 dBm)	745	67	1289	45	91.7%	94.3%	93.0%
Poor signal quality (RSSI < -80 dBm)	634	134	1012	89	82.6%	87.7%	85.1%
Mixed mobility (10% mobile nodes)	721	89	1334	67	89.0%	91.5%	90.2%
Energy constrained (50% low battery)	678	112	1245	78	85.8%	89.7%	87.7%
Overall average	734	85	1267	58	89.6%	92.4%	90.9%

6.6. Simulation results on lia's misbehavior detection accuracy

To evaluate the LIA component's ability to avoid *paranoia* (false positives) while maintaining adequate sensitivity (avoiding false negatives), we conducted a comprehensive set of experiments under representative network conditions. These conditions were selected to reflect typical IoT deployment scenarios and encompass both benign network behavior and intentional malicious actions.

The experiments were organized into two primary classes: benign link failures and malicious behavior. Benign failures correspond to normal network disruptions, such as variations in signal quality (high, medium, or poor RSSI), mixed node mobility (10% mobile nodes), or energy-constrained nodes (50% low battery). Malicious behavior was simulated through nodes performing disruptive actions, including dropping all incoming packets (blackhole attacks), selectively forwarding traffic (grayhole attacks), advertising false RPL ranks, or injecting false data. Each interaction was labeled with its ground truth (*Benign Failure* or *Malicious Behavior*) for subsequent evaluation of detection accuracy.

The LIA classifies packet transmissions using contextual reasoning triggered by failed transmissions (absence of acknowledgment). A failure is classified as *Benign* if either it occurs under weak signal conditions (RSSI < -75 dBm or LQI < 60) without significant routing anomalies (rank consistency > 70%), or if it originates from a low-energy node (below 20%) with a historically high success rate (above 80%). Conversely, a failure is classified as *Malicious* if it exhibits strong indications of intentional disruption, such as significant rank anomalies (consistency < 70%) or low forwarding success (below 30%). Failures that do not satisfy either criterion are labeled *Uncertain*, while successful transmissions are categorized as *Normal Operation*.

To systematically quantify the LIA's performance, we employed standard metrics: True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). These were used to compute Precision, Recall, and F1-Score, enabling rigorous evaluation of the trade-off between false positives and false negatives across the different network conditions represented in Table 13.

The results in Table 13 demonstrate the effectiveness of the LIA's contextual reasoning in achieving a crucial balance between false positive mitigation and false negative control. A primary strength of the LIA is its ability to significantly reduce the rate of false positives compared to a naive detection model that would classify all packet failures as malicious. This is evidenced by a low false positive rate of 1.6% under high signal quality conditions, where 1456 benign failures were correctly identified. Even as network conditions degraded, the system showed resilience, maintaining a manageable false positive rate of 11.7% under poor signal conditions. Across all tested scenarios, the LIA correctly classified between 82.6% and 97.5% of benign interactions, confirming its capacity to avoid unnecessary penalization of well-behaving nodes experiencing genuine communication issues.

Concurrently, the LIA maintains a high degree of sensitivity to actual malicious behavior, effectively controlling the false negative rate. The overall rate at which malicious activities were missed was 7.3%, indicating robust detection capability. Under optimal network conditions, the system's sensitivity peaked at 98.8%. While detection performance degrades gracefully under more adverse conditions, recall consistently remains above 87.7%. This combined performance demonstrates that the LIA's contextual reasoning successfully minimizes erroneous accusations against benign nodes without significantly compromising its ability to detect and respond to genuine malicious behavior.

To illustrate the tangible benefits of contextual reasoning, Table 14 compares the LIA's performance against a naive approach that classifies every packet failure as malicious. The naive approach achieves zero false negatives by labeling all failures as malicious, ensuring that no actual attacks are missed. However, this comes at a substantial cost: a high false positive rate of 34.7%, resulting in unnecessary isolation of well-behaving nodes and potential degradation of overall network performance. Consequently, the naive approach achieves a relatively low F1-score of 72.3%, reflecting the poor balance between precision and recall. In contrast, the LIA leverages contextual information to differentiate between benign failures and malicious behavior, reducing the false positive rate dramatically to 6.3%, while maintaining a modest false negative rate of 7.3%. Overall, the LIA achieves an F1-score of 90.9%, representing an 18.6% improvement over the naive approach. These results highlight the effectiveness of contextual reasoning in maintaining high detection accuracy while minimizing erroneous accusations, thereby improving both the robustness and reliability of the DeSIST framework.

Table 15 provides a sensitivity analysis of the LIA's RSSI and LQI thresholds, illustrating how different configurations impact the trade-off between false positives and false negatives. A higher RSSI or LQI threshold results in a more conservative detection strategy, reducing false positives at the expense of increased false negatives. For example, the configuration with RSSI = -70 dBm and LQI = 70 yields a low false positive rate of 3.2% but a relatively high false negative rate of 12.1%, indicating that some malicious behaviors may go undetected under overly cautious settings.

Conversely, lowering the thresholds to RSSI = -80 dBm and LQI = 50 produces an aggressive detection strategy, reducing the false negative rate to 4.1% while increasing false positives to 11.4%, which could penalize benign nodes unnecessarily. The middle

Table 14
Comparison of contextual vs. non-contextual detection.

Detection approach	FP rate	FN rate	F1-score
Naive (No context)	34.7%	0.0%	72.3%
LIA with context	6.3%	7.3%	90.9%
Improvement	-28.4%	+7.3%	+18.6%

Table 15
LIA threshold sensitivity analysis.

RSSI threshold (dBm)	LQI threshold	FP rate	FN rate	Optimal balance
-70	70	3.2%	12.1%	Conservative
-75	60	6.3%	7.3%	Balanced
-80	50	11.4%	4.1%	Aggressive

configuration, RSSI = -75 dBm and LQI = 60, achieves a balanced trade-off, maintaining a false positive rate of 6.3% and a false negative rate of 7.3%. This setting ensures robust detection of malicious activity while minimizing misclassification of benign nodes, making it suitable for typical IoT deployment scenarios.

These results demonstrate that careful tuning of LIA thresholds is essential to achieving optimal detection performance. By selecting thresholds that balance sensitivity and specificity, the LIA can maintain high accuracy across diverse network conditions while avoiding unnecessary disruptions to well-behaving nodes.

7. Conclusion and future work

The explosive growth of IoT ecosystems has underscored the urgent need for security mechanisms that are scalable, decentralized, and adaptive to heterogeneous environments. Conventional approaches often rely on centralized enforcement, explicit trust scores, or computationally expensive cryptographic protocols, which may be impractical for resource-constrained IoT devices. To address these limitations, this paper introduced DeSIST, a game-theoretic Zero Trust framework that reconceptualizes IoT security as a sequence of rational interactions among nodes. By aligning payoffs with cooperative outcomes and penalizing malicious or non-compliant behavior, DeSIST achieves emergent security while reducing dependence on heavyweight cryptographic measures.

DeSIST provides a lightweight mechanism enabling nodes to dynamically adapt their behavior through LIAs, allowing continuous evaluation of peers without maintaining complex trust histories. It employs a payoff structure such that cooperation becomes the most rational choice, while misbehavior leads to strategic isolation. The SDU uses this payoff structure to evaluate possible strategies and select the one that maximizes expected utility, fostering resilience against a wide range of prevalent IoT attack vectors.

Simulation results demonstrate significant security and performance improvements achieved by DeSIST compared to Standard RPL and the Representative TBS across all evaluated attack scenarios. The framework's primary strength lies in its ability to facilitate rapid, decentralized adaptation to malicious or undesirable behavior through utility-driven strategic decisions.

To further enhance the framework, future work will focus on the following areas:

- Ensuring expected utility calculations in the SDU remain feasible for highly constrained devices, particularly as game models scale in complexity.
- Strengthening resilience under irrational or bounded-rational node behavior, as well as defending against threats like Sybil attacks, collusion, and deviations from rational utility maximization.
- Managing scalability challenges in large networks with thousands of nodes to minimize coordination costs.
- Validating DeSIST on real IoT hardware to measure execution cycles, memory usage, and energy consumption, and to uncover practical implementation challenges such as timing variations, memory limitations, or communication delays.

Overall, DeSIST represents a novel step toward decentralized, incentive-driven IoT security. By leveraging game theory and rational payoff design, it removes reliance on central authorities and explicit trust models, allowing secure behavior to emerge naturally from strategic interactions. As IoT networks continue to expand and diversify, frameworks like DeSIST provide a foundation for developing adaptive, lightweight, and scalable security mechanisms that align the self-interest of individual devices with the collective resilience of the entire network.

CRediT authorship contribution statement

Seyed Hossein Ahmadpanah: Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Meghdad Mirabi:** Writing – review & editing, Visualization, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Conceptualization. **Sanaz Sobhanloo:** Writing – original draft, Visualization, Methodology, Formal analysis, Data curation. **Pania Afsharfarnia:** Methodology, Formal analysis, Data curation. **Donya Fallah:** Methodology, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The complete source code for the DeSIST framework is publicly available at <https://github.com/ahmadpanah/DeSIST>.

References

- [1] O. Said, M. Masud, Towards Internet of Things: Survey and future vision, *Int. J. Comput. Netw.* 5 (1) (2013) 1–17.
- [2] D. Mendez Mena, I. Papapanagiotou, B. Yang, Internet of Things: Survey on security, *Inf. Secur. J. Glob. Perspect.* 27 (3) (2018) 162–182.
- [3] J. Gubbi, R. Buyya, S. Marusic, M. Palaniswami, Internet of Things (IoT): A vision, architectural elements, and future directions, *Future Gener. Comput. Syst.* 29 (7) (2013) 1645–1660.
- [4] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, M. Ayyash, Internet of Things: A survey on enabling technologies, protocols, and applications, *IEEE Commun. Surv. Tutor.* 17 (4) (2015) 2347–2376.
- [5] G. Gunawan, W. Shabrina, W. Andriani, Systematic literature review implementation of the internet of things (IoT) in smart city development, *Buana Inf. Technol. Comput. Sci. (Bit Cs)* 3 (2022) 41–46.
- [6] E. Yaacoub, M. Alouini, Efficient fronthaul and backhaul connectivity for IoT traffic in rural areas, *IEEE Internet Things Mag.* 4 (2021) 60–66.
- [7] Z. Nain, A. Musaddiq, Y. Qadri, A. Nauman, M. Afzal, S. Kim, Riata: A reinforcement learning-based intelligent routing update scheme for future generation IoT networks, *IEEE Access* 9 (2021) 81161–81172.
- [8] R. Hajian, S. ZakeriKia, S.H. Erfani, M. Mirabi, SHAPARAK: Scalable healthcare authentication protocol with attack-resilience and anonymous key-agreement, *Comput. Netw.* 183 (2020) 107567.
- [9] Y. Shen, C. Shepherd, C.M. Ahmed, S. Shen, S. Yu, SGD3QN: Joint stochastic games and dueling double deep Q-networks for defending malware propagation in edge intelligence-enabled Internet of Things, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 6978–6990.
- [10] Y. Shen, C. Shepherd, C.M. Ahmed, S. Shen, S. Yu, Integrating deep spiking Q-network into hypergame-theoretic deceptive defense for mitigating malware propagation in Edge Intelligence-enabled IoT Systems, *IEEE Trans. Serv. Comput.* (2025) 1487–1499.
- [11] R. Lakhani, Cybersecurity threats in Internet of Things (IoT) networks: Vulnerabilities and defense mechanisms, *Int. J. Eng. Comput. Sci.* (2023) 25965–25980.
- [12] A.S. Alavizadeh, S.H. Erfani, M. Mirabi, A. Sahafi, An efficient distributed and secure algorithm for transaction confirmation in IOTA using cloud computing, *J. Supercomput.* 80 (2) (2024) 1491–1521.
- [13] K. Liu, J. Liu, J. Li, X. Jiang, TS-IDF: A two-stage intrusion detection framework for complex maritime IoT, in: *The 5th International Conference on Computer, Big Data and Artificial Intelligence, ICCBD+AI, 2024*, pp. 654–658.
- [14] A.S. Alavizadeh, S.H. Erfani, M. Mirabi, A. Sahafi, A distributed reliable collusion-free algorithm for selecting multiple coordinators in IOTA using fog computing, *Concurr. Comput. Pr. Exp.* 11 (2024) e8009.
- [15] S. Yu, X. Wang, Y. Shen, G. Wu, S. Yu, S. Shen, Novel intrusion detection strategies with optimal hyper parameters for industrial internet of things based on stochastic games and double deep Q-networks, *IEEE Internet Things J.* 11 (17) (2024) 29132–29145.
- [16] B.B. Zarpelão, R.S. Miani, C.T. Kawakani, S.C. De Alvarenga, A survey of intrusion detection in Internet of Things, *J. Netw. Comput. Appl.* 84 (2017) 25–37.
- [17] S.T. Bakhsh, S. Alghamdi, R.A. Alsemmeari, S.R. Hassan, An adaptive intrusion detection and prevention system for Internet of Things, *Int. J. Distrib. Sens. Netw.* 15 (11) (2019).
- [18] S. Shen, L. Xie, Y. Zhang, G. Wu, H. Zhang, S. Yu, Joint differential game and double deep Q-networks for suppressing malware spread in Industrial Internet of Things, *IEEE Trans. Inf. Forensics Secur.* 18 (2023) 5302–5315.
- [19] Y.B. Saied, A. Olivereau, D. Zeglache, M. Laurent, Trust management system design for the Internet of Things: A context-aware and multi-service approach, *Comput. Secur.* 39 (2013) 351–365.
- [20] Z. Yan, P. Zhang, A.V. Vasilakos, A survey on trust management for Internet of Things, *J. Netw. Comput. Appl.* 42 (2014) 120–134.
- [21] I.U. Din, M. Guizani, B.-S. Kim, S. Hassan, M.K. Khan, Trust management techniques for the Internet of Things: A survey, *IEEE Access* 7 (2018) 29763–29787.
- [22] M.A. Azad, S. Bag, F. Hao, A. Shalaginov, Decentralized self-enforcing trust management system for social Internet of Things, *IEEE Internet Things J.* 7 (4) (2020) 2690–2703.
- [23] S. Yu, R. Zhai, Y. Shen, G. Wu, H. Zhang, S. Yu, S. Shen, Deep Q-network-based open-set intrusion detection solution for industrial Internet of Things, *IEEE Internet Things J.* 11 (7) (2023) 12536–12550.
- [24] Y. He, D. Huang, L. Chen, Y. Ni, X. Ma, A survey on zero trust architecture: Challenges and future trends, *Wirel. Commun. Mob. Comput.* 2022 (1) (2022) 6476274.
- [25] S. Teerakanok, T. Uehara, A. Inomata, Migrating to zero trust architecture: Reviews and challenges, *Secur. Commun. Netw.* 2021 (1) (2021) 9947347.
- [26] N.F. Syed, S.W. Shah, A. Shaghghi, A. Anwar, Z. Baig, R. Doss, Zero trust architecture (ZTA): A comprehensive survey, *IEEE Access* 10 (2022) 57143–57179.
- [27] M. Tsai, S. Lee, S.W. Shieh, Strategy for implementing of zero trust architecture, *IEEE Trans. Reliab.* 73 (1) (2024) 93–100.
- [28] S. Roy, C. Ellis, S. Shiva, D. Dasgupta, V. Shandilya, Q. Wu, A survey of game theory as applied to network security, in: *The 43rd Hawaii International Conference on System Sciences*, 2010, pp. 1–10.
- [29] X. Liang, Y. Xiao, Game theory for network security, *IEEE Commun. Surv. Tutor.* 15 (1) (2012) 472–486.
- [30] C. Chi, Y. Wang, X. Tong, M. Siddula, Z. Cai, Game theory in Internet of Things: A survey, *IEEE Internet Things J.* 9 (14) (2021) 12125–12146.
- [31] M.P. Wellman, K. Tuyls, A. Greenwald, Empirical game theoretic analysis: A survey, *J. Artificial Intelligence Res.* 82 (2025) 1017–1076.
- [32] S. Shen, C. Cai, Y. Shen, X. Wu, W. Ke, S. Yu, Joint mean-field game and multiagent asynchronous advantage actor-critic for edge intelligence-based IoT malware propagation defense, *IEEE Trans. Dependable Secur. Comput.* (2025) 3824–3838.
- [33] T. Winter, P. Thubert, A. Brandt, J. Hui, R. Kelsey, P. Levis, K. Pister, R. Struik, J.P. Vasseur, R. Alexander, RFC 6550: RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks, 2012.
- [34] A.J. Hintaw, S. Manickam, M.F. Aboalmaalay, S. Karuppiah, MQTT vulnerabilities, attack vectors and solutions in the Internet of Things (IoT), *IETE J. Res.* 69 (6) (2023) 3368–3397.
- [35] N. Torres, P. Pinto, S.I. Lopes, Security vulnerabilities in LPWANs—an attack vector analysis for the IoT ecosystem, *Appl. Sci.* 11 (7) (2021) 3176.
- [36] N. Mishra, S. Pandya, Internet of Things applications, security challenges, attacks, intrusion detection, and future visions: A systematic review, *IEEE Access* 9 (2021) 59353–59377.

- [37] S. Prajapati, A. Singh, Cyber-attacks on Internet of Things (IoT) devices, attack vectors, and remedies: A position paper, in: *IoT and Cloud Computing for Societal Good*, 2021, pp. 277–295.
- [38] V. Bakhtyari, M. Mirabi, A. Salajegheh, S.H. Erfani, Combo-Chain: Towards a hierarchical attribute-based access control system for IoT with smart contract and sharding technique, *Internet Things* 25 (2024) 101080.
- [39] A.K. Sangaiah, A. Javadpour, F. Ja'fari, P. Pinto, H. Ahmadi, W. Zhang, CL-MLSP: The design of a detection mechanism for sinkhole attacks in smart cities, *Microprocess. Microsyst.* 90 (2022) 104504.
- [40] P.P.-Y. Wu, F. Ruggeri, D. Mengersen, Dependent Cluster Mapping (DCMAP): Optimal clustering of directed acyclic graphs for statistical inference, 2023, arXiv preprint arXiv:2308.03970.
- [41] S. Mangelkar, S.N. Dhage, A.V. Nimkar, A comparative study on RPL attacks and security solutions, in: *The International Conference on Intelligent Computing and Control, I2C2*, 2017, pp. 1–6.
- [42] A. Raoof, A. Matrawy, C.-H. Lung, Routing attacks and mitigation methods for RPL-based Internet of Things, *IEEE Commun. Surv. Tutor.* 21 (2) (2018) 1582–1606.
- [43] V.R. Rajasekar, S. Rajkumar, A study on impact of DIS flooding attack on RPL-based 6LowPAN network, *Microprocess. Microsyst.* 94 (2022) 104675.
- [44] A. Verma, V. Ranga, Addressing flooding attacks in IPv6-based low power and lossy networks, in: *TENCON 2019-2019 IEEE Region 10 Conference, TENCON*, 2019, pp. 552–557.
- [45] L. Prospero, R. Costa, L. Badia, Resource sharing in the Internet of Things and selfish behaviors of the agents, *IEEE Trans. Circuits Syst. Express Briefs* 68 (12) (2021) 3488–3492.
- [46] G.H. Abdi, A.H.R. Sheikhan, S. Kordrostami, A. Ghane, S. Babaie, A novel selfish node detection based on reputation and game theory in Internet of Things, *Computing* 106 (1) (2024) 81–107.
- [47] L. Yang, C. Ding, M. Wu, K. Wang, Robust detection of false data injection attacks for data aggregation in an Internet of Things-based environmental surveillance, *Comput. Netw.* 129 (2017) 410–428.
- [48] M.M.N. Aboelwafa, K.G. Seddik, M.H. Eldefrawy, Y. Gadallah, M. Gidlund, A machine-learning-based technique for false data injection attacks detection in industrial IoT, *IEEE Internet Things J.* 7 (9) (2020) 8462–8471.
- [49] S. Rose, Planning for a Zero Trust Architecture: A Planning Guide for Federal Administrators, 2022.
- [50] S. Li, M. Iqbal, N. Saxena, Future industry internet of things with zero-trust security, *Inf. Syst. Front.* 26 (5) (2024) 1653–1666.
- [51] N. Bhattacharya, D. Bansal, Enabling secure and efficient healthcare delivery in smart cities through 6G infrastructure, in: *Building Tomorrow's Smart Cities with 6G Infrastructure Technology*, 2025, pp. 365–398.
- [52] S. Zangaraki, M. Mirabi, S.H. Erfani, A. Sahafi, SecShield: An IoT access control framework with edge caching using software defined network, *Peer-To-Peer Netw. Appl.* 18 (1) (2025) 56.
- [53] J. Barach, Towards zero trust security in SDN: A multi-layered defense strategy, in: *The 26th International Conference on Distributed Computing and Networking*, 2025, pp. 331–339.
- [54] S. Shakya, R. Abbas, S. Maric, A novel zero-touch, zero-trust, AI/ML enablement framework for IoT network security, 2025, arXiv preprint arXiv:2502.03614.
- [55] H. Mahmoud, J. Arshad, A. Aneiba, A systematic review of blockchain-based privacy-preserving reputation systems for IoT applications, *Distrib. Ledger Technol.: Res. Pr.* 3 (4) (2025) 1–40.
- [56] A.H. Langaroudi, M. Abdollahi Azgomi, A collusion-resistant trust management system in fog computing, *J. Supercomput.* 81 (8) (2025) 1–40.
- [57] K. Hayawi, I. Makhdoom, S. Khalid, R.A. Ikuesan, M. Kaosar, I. Ahmad, A false positive resilient distributed trust management framework for collaborative intrusion detection systems, *IEEE Trans. Serv. Comput.* (2025).
- [58] M. Beynon, B. Curry, P. Morgan, The Dempster-Shafer theory of evidence: An alternative approach to multicriteria decision modelling, *Omega* 28 (1) (2000) 37–50.
- [59] R. Kait, S. Kaur, P. Sharma, C. Ankita, T. Kumar, X. Cheng, Fuzzy logic-based trusted routing protocol using vehicular cloud networks for smart cities, *Expert Syst.* 42 (1) (2025) e13561.
- [60] S. Higashi, P. Ampirit, E. Qafzezi, M. Ikeda, K. Matsuo, L. Barolli, A fuzzy-based system for assessment of relational trust in IoT and social networks, *Internet Things* (2025) 101528.
- [61] Y. Otoum, A. Asad, A. Nayak, Blockchain meets adaptive honeypots: A trust-aware approach to Next-Gen IoT security, 2025, arXiv preprint arXiv:2504.16226.
- [62] R. Ratnayake, M. Liyanage, L. Murphy, Evaluating data trust in blockchain-based IoT systems using machine learning techniques, in: *The IEEE 22nd Consumer Communications & Networking Conference, CCNC*, 2025, pp. 1–6.
- [63] M. Hadian, S.H. Erfani, M. Deypir, M. Mirabi, CD-TMS: A combinatorial design-based token management system to enhance security and performance in blockchain, *Clust. Comput.* 27 (4) (2024) 4515–4536.
- [64] M. Danish, S. Shahid, A. Ghafar, K. Hamid, N. Ali, A. Ghani, S.A. Mandan, M. Ibrar, Security of next-generation networks: A hybrid approach using ML-algorithm and game theory with SDWSN, *Annu. Methodol. Arch. Res. Rev.* 3 (4) (2025) 18–36.
- [65] K.C. Chaganti, A scalable, lightweight AI-driven security framework for IoT ecosystems: Optimization and game theory approaches, *IEEE Access* (2025).
- [66] C.A. Holt, A.E. Roth, The Nash equilibrium: A perspective, in: *Proceedings of the National Academy of Sciences*, 2004, pp. 3999–4002.
- [67] F. Razvan, C. Mitica, Enhancing network security through integration of game theory in software-defined networking framework, *Int. J. Inf. Secur.* 24 (3) (2025) 100.
- [68] H. Sun, C. Shao, J. Zhang, K. Wang, W. Huang, Evolution analysis of network attack and defense situation based on game theory., *Comput. Mater. Contin.* 83 (1) (2025).
- [69] L. SaiRamesh, G.J.W. Kathrine, V. Sathiyavathi, K. Selvakumar, S. Sabena, Intelligent intrusion detection and prevention system for IoT using game theoretic approach, *Wirel. Pers. Commun.* (2025) 1–16.
- [70] P. Yao, B. Yan, Q. Yang, Game theoretical decision-making of dynamic defense in cyber-physical power systems under cyber-attacks, *ACM Trans. Cyber-Physical Syst.* 9 (2) (2025) 1–21.
- [71] S. Gupta, N. Sharma, A novel cluster based reliable security enhancement in FANET directed by game theory, *Soft Comput.* (2025) 1–16.
- [72] S.A. Almunasher, M.J.F. Alenazi, EEPs: Optimizing energy-efficient path selection in the Internet of Battlefield Things (IoBT) utilizing SDN, *Peer-To-Peer Netw. Appl.* 18 (1) (2025) 1–23.
- [73] B.S.N. Murthy, V. Madhusudan, M.N. Srinivas, L. Guerrini, A. Zeb, N.-N. Dao, S. Cho, Analytical optimal control of delayed worm propagation model in heterogeneous IoT systems, *IEEE Access* (2025).
- [74] X. Ma, M. Gao, “Lure the enemy in deep”: Confronting rogue UAV through diverse hybrid jamming, *IEEE Access* 13 (2025) 68351–68369.
- [75] A. Navaei Tourani, H. Haj Seyyed Javadi, H. Navidi, A. Sharifi, A study on trust-rating mechanism for WSN node sensors using evolutionary game theory, *J. Supercomput.* 81 (2) (2025) 401.
- [76] B. Kumar, B. Bhuyan, Game theoretic defense framework against Sybil attacks, *SN Comput. Sci.* 5 (7) (2024) 857.
- [77] A. Cirne, P.R. Sousa, J.S. Resende, L. Antunes, Hardware security for Internet of Things identity assurance, *IEEE Commun. Surv. Tutor.* 26 (2) (2024) 1041–1079.
- [78] S. Banerjee, H. Konishi, T. Sönmez, Core in a simple coalition formation game, *Soc. Choice Welf.* 18 (1) (2001) 135–153.
- [79] J.P. Kahan, A. Rapoport, *Theories of Coalition Formation*, Psychology Press, 2014.
- [80] H. Konishi, D. Ray, Coalition formation as a dynamic process, *J. Econom. Theory* 110 (1) (2003) 1–41.

- [81] G. Owen, *Game Theory*, Emerald Group Publishing, 2013.
- [82] R. Branzei, D. Dimitrov, S. Tijs, *Models in Cooperative Game Theory*, Springer, 2008.
- [83] S. Lozano, P. Moreno, B. Adenso-Díaz, E. Algaba, Cooperative game theory approach to allocating benefits of horizontal cooperation, *European J. Oper. Res.* 229 (2) (2013) 444–452.
- [84] C.M. Kahn, D. Mookherjee, The good, the bad, and the ugly: Coalition proof equilibrium in infinite games, *Games Econom. Behav.* 4 (1) (1992) 101–121.
- [85] B.D. Bernheim, B. Peleg, M.D. Whinston, Coalition-proof nash equilibria I. concepts, *J. Econom. Theory* 42 (1) (1987) 1–12.
- [86] D. Moreno, J. Wooders, Coalition-proof equilibrium, *Games Econom. Behav.* 17 (1) (1996) 80–112.
- [87] D. Zinoviev, *Discrete event simulation: It's easy with SimPy!*, 2024, ArXiv preprint. [arXiv:2405.01562](https://arxiv.org/abs/2405.01562).
- [88] S.M.Z. Khan, M.M. Alam, Y. Le Moullec, A. Kuusik, S. Päränd, C. Verikoukis, An empirical modeling for the baseline energy consumption of an NB-IoT radio transceiver, *IEEE Internet Things J.* 8 (19) (2021) 14756–14772.
- [89] Y.-W. Ma, P.-H. Chiu, A novel risk-based access control engine in zero trust architecture for IoT network, *Int. J. Inf. Secur.* 24 (3) (2025) 124.